

Angular Learning Series

Topics 21 to 26

Updated Modern Syntax Edition

Contents

Topic 21 -- takeUntilDestroyed and DestroyRef

Topic 22 -- async Pipe

Topic 23 -- Angular Signals (signal, computed, effect)

Topic 24 -- OnPush Change Detection

Topic 25 -- Component Lifecycle Hooks

Topic 26 -- input() and output() -- Component Communication in Depth

Generated: June 2026

Topic 21 -- takeUntilDestroyed and DestroyRef

0. Difficulty and Priority Rating

Difficulty: Intermediate -- the API is simple; understanding injection context rules and when to use toSignal vs takeUntilDestroyed takes a few real examples

Angular Relevance: Critical -- this is the modern solution to the subscription memory leak problem from Topic 20; every Angular 16+ app should use this pattern as its default cleanup mechanism

1. Explanation

The Smart Power Strip Analogy

Imagine you have a desk with ten devices plugged in -- monitor, laptop, phone charger, desk lamp, speakers. When you leave the office every night you have to physically unplug each one. Every. Single. Night.

Then someone installs a smart power strip that connects to your building's security system. Now when you badge out, the power strip automatically cuts power to everything on your desk. You never forget to unplug something. You never come back to a device that was on all night.

DestroyRef is that smart power strip. Connect it once. Every device (subscription) plugged into it (via takeUntilDestroyed) powers off automatically when you leave (component destroyed).

The old pattern (takeUntil + destroy\$ Subject) was you physically unplugging each device. DestroyRef is the automated system that does it for you -- impossible to forget, no boilerplate.

In Plain English

- * DestroyRef = an injectable object that knows exactly when the current component/service/directive is destroyed. It has one method: onDestroy(callback) -- register any function to run on destroy
- * takeUntilDestroyed(destroyRef) = an RxJS operator that completes an Observable when DestroyRef fires -- the smart power strip connection for subscriptions
- * Injection context = where Angular can resolve inject() calls -- field initialisers and constructors. NOT lifecycle hooks like ngOnInit
- * toSignal() = converts an Observable to a Signal, with DestroyRef managed internally by Angular -- the ultimate zero-boilerplate pattern

The Key Rule -- Injection Context

```
Field initialiser inject() works here

Constructor inject() works here

ngOnInit inject() does NOT work here

ngOnChanges not an injection context

Any other method not an injection context
```

Rule: inject DestroyRef in a field, then pass it explicitly anywhere

2. Connection to Prior Concepts

This topic is the direct solution to Topic 20's core problem:

* Topic 20 (RxJS Subscription Management) -- identified the problem: unmanaged subscriptions cause memory leaks. Topic 21 gives you the modern Angular solution -- DestroyRef + takeUntilDestroyed

* Topic 3 (Closures) -- takeUntilDestroyed works by closing over the DestroyRef -- when the component is destroyed, the closure fires and the Observable completes

* Topic 5 (Event Loop) -- DestroyRef.onDestroy() callbacks run synchronously during Angular's destroy cycle -- before the component is fully garbage collected

* Topic 23 (Signals -- coming next) -- toSignal() uses DestroyRef internally. Understanding DestroyRef explains why toSignal() works without explicit subscription management

Bridging note: Topic 20 showed you the problem and three solutions. Topic 21 goes deep on the best of those solutions -- DestroyRef -- and adds toSignal() as the next level up.

3. Code Example

Example 1 -- What DestroyRef actually is -- under the hood

Before using it, see what it actually does -- it is just a notification system.

```
// DestroyRef is an injectable -- get it like any other service

import { DestroyRef, inject, Component, OnInit } from '@angular/core';

@Component({ selector: 'app-demo', standalone: true, template: '...' })
export class DemoComponent implements OnInit {

  // inject() in field initialiser = injection context
  private readonly destroyRef = inject(DestroyRef);

  ngOnInit(): void {

    // DestroyRef has ONE method: onDestroy(callback)

    // Register ANY cleanup logic -- runs when this component is destroyed
    this.destroyRef.onDestroy(() => {

      console.log('DemoComponent is being destroyed -- cleanup running');

      clearInterval(this.pollingTimer); // clear a timer

      this.socket?.close(); // close a WebSocket

      this.thirdPartyLib?.dispose(); // clean up a third-party lib

      localStorage.removeItem('temp'); // clean up temporary storage

    });

    // Every destroyRef.onDestroy() call adds ANOTHER callback

    // All of them run when the component is destroyed
```

```

this.destroyRef.onDestroy(() => {

  console.log('Second cleanup callback -- also runs on destroy');

});

}

}

```

Example 2 -- takeUntilDestroyed -- three ways to use it

The injection context rule is the most important thing to understand here.

```

import { takeUntilDestroyed } from '@angular/core/rxjs-interop';
import { DestroyRef, inject } from '@angular/core';

// WAY 1 -- Explicit destroyRef -- works ANYWHERE in the class
// Most flexible -- recommended default

@Component({ selector: 'app-way1', standalone: true, template: '...' })
export class Way1Component implements OnInit {

  private readonly destroyRef = inject(DestroyRef);
  private readonly dataService = inject(DataService);

  ngOnInit(): void {

    // destroyRef passed explicitly -- works in ngOnInit, methods, anywhere
    this.dataService.getData().pipe(
      takeUntilDestroyed(this.destroyRef)
    ).subscribe(data => { this.data = data; });

  }

}

// WAY 2 -- No argument -- field initialiser injection context
// Clean, concise -- but subscription must be set up in the field

@Component({ selector: 'app-way2', standalone: true, template: '...' })
export class Way2Component {

```

```

private readonly dataService = inject(DataService);

// Field initialiser = injection context

// takeUntilDestroyed() with no argument auto-detects destroyRef

private readonly data$ = this.dataService.getData().pipe(
  takeUntilDestroyed() // injection context -- auto-detects
);
}

// WRONG -- ngOnInit is NOT an injection context

@Component({ selector: 'app-wrong', standalone: true, template: '...' })
export class WrongComponent implements OnInit {

  private readonly dataService = inject(DataService);

  ngOnInit(): void {
    this.dataService.getData().pipe(
      takeUntilDestroyed() // Error: not in injection context
    ).subscribe(data => { this.data = data; });
  }
}

// Fix: either use Way 1 (explicit destroyRef)
// or move subscription to constructor/field initialiser

```

Example 3 -- DestroyRef for non-subscription cleanup

```

@Injectable({ providedIn: 'root' })
export class WebSocketService {

  private readonly destroyRef = inject(DestroyRef);

  private socket: WebSocket | null = null;

  connect(url: string): Observable(MessageEvent) {

    this.socket = new WebSocket(url);
  }
}

```

```

this.destroyRef.onDestroy(() => {

this.socket?.close();

this.socket = null;

console.log('WebSocket closed on service destroy');

});

return new Observable(observer => {

this.socket!.onmessage = event => observer.next(event);

this.socket!.onerror = err => observer.error(err);

this.socket!.onclose = () => observer.complete();

});

}

}

```

Angular Example -- toSignal -- DestroyRef working invisibly

```

import { toSignal } from '@angular/core/rxjs-interop';

import { inject, signal, computed } from '@angular/core';

@Component({

selector: 'app-product-list',

standalone: true,

template: `

@if (isLoading()) {

(div)Loading products...(/div)

}

@for (product of filteredProducts(); track product.id) {

(div){{{ product.name }} -- {{ product.price.toFixed(2) }}(/div)

}

(p)Showing {{ filteredProducts().length }} of {{ products().length }}(/p)

`

})

export class ProductListComponent {

```

```

private readonly productService = inject(ProductService);

// toSignal -- converts Observable to Signal

// DestroyRef managed INTERNALLY by Angular -- zero boilerplate

readonly products = toSignal(
  this.productService.getProducts(),
  { initialValue: [] as Product[] }
);

readonly searchTerm = signal('');

readonly filteredProducts = computed(() => {
  const term = this.searchTerm().toLowerCase();
  return this.products().filter(p =>
    p.name.toLowerCase().includes(term)
  );
});

// No ngOnDestroy, no takeUntilDestroyed, no subscribe management
}

```

4. Before and After Refactor

The Problem in Plain English:

Before DestroyRef and takeUntilDestroyed, every component needed a destroy\$ Subject, an ngOnDestroy hook, and two lines of cleanup code -- even when the component was otherwise simple. It was unavoidable boilerplate. Forgetting any part caused a memory leak. DestroyRef collapses all of that into one injected object and one operator.

```

// BEFORE -- Angular 15 and earlier -- mandatory boilerplate

@Component({ selector: 'app-user-dashboard', standalone: true, template: '...' })

export class UserDashboardComponent implements OnInit, OnDestroy {

  userData: User | null = null;

  notifications: Notification[] = [];

  recentOrders: Order[] = [];
}

```

```

isLoading = false;

// Boilerplate 1 -- destroy Subject

private readonly destroy$ = new Subject<void>();

private readonly userService = inject<UserService>();
private readonly notifService = inject<NotificationService>();
private readonly orderService = inject<OrderService>();

ngOnInit(): void {
  this.isLoading = true;

  this.userService.getCurrentUser().pipe(
    takeUntil(this.destroy$)
  ).subscribe(user => { this.userData = user; });

  this.notifService.getNotifications().pipe(
    takeUntil(this.destroy$)
  ).subscribe(notif => { this.notifications = notif; });

  this.orderService.getRecentOrders().pipe(
    takeUntil(this.destroy$)
  ).subscribe(orders => {
    this.recentOrders = orders;
    this.isLoading = false;
  });
}

// Boilerplate 2 -- ngOnDestroy with two lines
ngOnDestroy(): void {
  this.destroy$.next(); // fire the signal
  this.destroy$.complete(); // clean up Subject
}
}

```



```

// AFTER -- Angular 16+ with DestroyRef + takeUntilDestroyed + toSignal

@Component({ selector: 'app-user-dashboard', standalone: true, template: '...' })
export class UserDashboardComponent implements OnInit {

  private readonly userService = inject(UserService);
  private readonly notifService = inject(NotificationService);
  private readonly orderService = inject(OrderService);

  // toSignal -- zero subscription management for pure data
  readonly userData = toSignal(
    this.userService.getCurrentUser(),
    { initialValue: null }
  );
  readonly notifications = toSignal(
    this.notifService.getNotifications(),
    { initialValue: [] as Notification[] }
  );

  recentOrders: Order[] = [];
  isLoading = false;

  // One line -- no Subject, no ngOnDestroy
  private readonly destroyRef = inject(DestroyRef);

  ngOnInit(): void {
    this.isLoading = true;

    this.orderService.getRecentOrders().pipe(
      takeUntilDestroyed(this.destroyRef)
    ).subscribe(orders => {
      this.recentOrders = orders;
      this.isLoading = false;
    });
  }
}

```

```
});

}

// No ngOnDestroy at all

// No destroy$ Subject

// No boilerplate

}
```

5. Common Mistakes and Misconceptions

1. "DestroyRef is just the new name for ngOnDestroy -- same thing"

They are related but meaningfully different. `ngOnDestroy` is a lifecycle hook you implement -- called by Angular when the component is destroyed. `DestroyRef` is an injectable you can pass around and register multiple callbacks on. The key difference: `DestroyRef` can be passed to functions and utilities outside the component -- `ngOnDestroy` cannot.

```
// DestroyRef -- injectable, passable, flexible

export class MyComponent {

  private readonly destroyRef = inject(DestroyRef);

  private readonly dataService = inject(DataService);

  private readonly helperService = inject(HelperService);

  ngOnInit(): void {

    // Pass destroyRef to a utility function OUTSIDE the component

    setupPolling(this.dataService, this.destroyRef);

    this.helperService.registerWithCleanup(this.destroyRef);

  }

}

// Utility function outside the component -- receives destroyRef

function setupPolling(service: DataService, destroyRef: DestroyRef): void {

  timer(0, 30_000).pipe(

    switchMap(() => service.getData()),

    takeUntilDestroyed(destroyRef) // uses the component's destroyRef

  ).subscribe(data => console.log(data));

}
```

```
}
```

2. "takeUntilDestroyed() without an argument always works"

Only works in an injection context -- field initialisers and constructors. `ngOnInit`, `ngOnChanges`, event handlers, and any method called after construction are NOT injection contexts.

```
// Fix 1 -- inject destroyRef in field, pass it explicitly

@Component({ ... })

export class MyComponent implements OnInit {

  private readonly destroyRef = inject(DestroyRef);

  private readonly dataService = inject(DataService);

  ngOnInit(): void {

    this.dataService.getData().pipe(

      takeUntilDestroyed(this.destroyRef) // explicit -- always works

    ).subscribe();

  }

}
```

3. "toSignal is just a shortcut for async pipe -- same result"

They solve the same problem differently. `async pipe` lives in the template -- subscribes when the component renders. `toSignal` lives in the class -- subscribes immediately, gives you a `Signal` usable in `computed()` values.

6. Do's and Don'ts

Inject `DestroyRef` as a class field -- always an injection context

Do NOT call `inject(DestroyRef)` inside `ngOnInit` or any lifecycle hook

Pass `destroyRef` explicitly to `takeUntilDestroyed` when in `ngOnInit`

Do NOT use `takeUntilDestroyed()` without argument outside a field or constructor

Use `toSignal()` for Observables bound to templates -- zero boilerplate

Do NOT use `destroy$ = new Subject(void)() + takeUntil` in new Angular 16+ code

Pass `destroyRef` to utility functions -- promotes clean separation

Do NOT re-implement the `destroy$ Subject` pattern -- it is obsolete in Angular 16+

Register non-subscription cleanup via `destroyRef.onDestroy()`

Do NOT implement `ngOnDestroy` just for subscription cleanup -- use `takeUntilDestroyed`

Use `toSignal()` with `initialValue` to avoid undefined during first render

Do NOT use `toSignal()` without `initialValue` and not handle the undefined case

7. Debugging Tips

Bug: Error -- takeUntilDestroyed() can only be used within an injection context

Plain English cause: takeUntilDestroyed() with no argument was called inside ngOnInit, a method, or an event handler -- not in a field initialiser or constructor

Fix: Inject DestroyRef as a class field and pass it explicitly: takeUntilDestroyed(this.destroyRef)

Bug: Component still leaks after adding takeUntilDestroyed

Plain English cause: takeUntilDestroyed was added to some subscriptions but not all -- one was missed

Fix: Search the component for every .subscribe() call and verify every one has takeUntilDestroyed in its pipe chain

Bug: toSignal throws Error: injection context at runtime

Plain English cause: toSignal() was called inside ngOnInit or a method -- not at class field level

Fix: Move the toSignal() call to a class field initialiser: readonly data = toSignal(this.service.getData(), { initialValue: [] })

8. Real-World Applications

Application 1 -- Composable Subscription Utility

Plain English first: As an Angular app grows, the same subscription patterns repeat -- polling, WebSocket connections, form value streams. Instead of copy-pasting takeUntilDestroyed(this.destroyRef) everywhere, extract these patterns into utility functions that accept a DestroyRef. This is DestroyRef's killer feature -- you can pass it to anything.

```
// Generic polling utility -- works for any Observable

function createPollingStream(
  source$: Observable(T),
  intervalMs: number,
  destroyRef: DestroyRef
): Observable(T) {
  return timer(0, intervalMs).pipe(
    switchMap(() => source$),
    takeUntilDestroyed(destroyRef)
  );
}

// Components use utilities cleanly

@Component({ selector: 'app-dashboard', standalone: true, template: '...' })
export class DashboardComponent implements OnInit {

  stats: DashboardStats | null = null;
```

```

orders: Order[] = [];

private readonly destroyRef = inject(DestroyRef);
private readonly statsService = inject(StatsService);
private readonly orderService = inject(OrderService);

ngOnInit(): void {
  createPollingStream(
    this.statsService.getStats(),
    30_000,
    this.destroyRef
  ).subscribe(stats => { this.stats = stats; });

  createPollingStream(
    this.orderService.getRecentOrders(),
    60_000,
    this.destroyRef
  ).subscribe(orders => { this.orders = orders; });
}
}

```

Application 2 -- Component-Scoped Service Cleanup

```

@Injectable() // No providedIn -- provided at component level

export class ComponentCacheService {

  private readonly cache = new Map();
  private readonly destroyRef = inject(DestroyRef);

  constructor() {

    this.destroyRef.onDestroy(() => {

      this.cache.clear();

      console.log('ComponentCacheService cleaned up');

    });
  }
}

```

```

set(key: string, value: unknown): void { this.cache.set(key, value); }

get(key: string): unknown { return this.cache.get(key); }

}

// Component provides the service -- service manages its own lifecycle

@Component({
  selector: 'app-product-detail',
  standalone: true,
  providers: [ComponentCacheService],
  template: '...'
})

export class ProductDetailComponent implements OnInit {

  private readonly cache = inject(ComponentCacheService);
  private readonly productService = inject(ProductService);
  product: Product | null = null;

  ngOnInit(): void {
    const cached = this.cache.get('current-product') as Product | undefined;
    if (!cached) {
      this.productService.getProduct(this.productId).subscribe(product => {
        this.cache.set('current-product', product);
        this.product = product;
      });
    } else {
      this.product = cached;
    }
  }
}

```

9. Practice Exercises

Exercise 1 -- Beginner

Create a `ClockComponent` that shows the current time, updating every second using `interval(1000)`. Implement it first without cleanup -- observe the memory leak by navigating away and checking the console. Then add `inject(DestroyRef)` and `takeUntilDestroyed`. Verify the interval stops firing after navigation.

Exercise 2 -- Intermediate

Build a `LiveSearchComponent` with a text input, a `debounceTime(300)` pipe, and a `searchService.search(query)` HTTP call using `switchMap`. Use `takeUntilDestroyed` for the subscription. Then convert the results to a `Signal` using `toSignal()` and verify you can derive a `resultCount` computed signal from it.

Exercise 3 -- Advanced

Extract all subscription logic from a `DashboardComponent` (polling stats, live notifications, user stream) into a `DashboardStreamService` scoped to the component (not providedIn: 'root'). The service must use `inject(DestroyRef)` to register its own cleanup and expose its streams via `toSignal()`. The component should contain zero `.subscribe()` calls and zero `ngOnDestroy`. Verify by checking that all streams stop when the component is destroyed.

10. Key Takeaways

Core Concepts in Plain English:

- * `DestroyRef` = the smart power strip -- injectable, one instance per component, fires on destroy
- * `takeUntilDestroyed(destroyRef)` = plugs a subscription into the smart power strip
- * Injection context = where `inject()` works -- field initialisers and constructors only -- NOT lifecycle hooks
- * Always inject `DestroyRef` as a class field -- then pass explicitly to `takeUntilDestroyed` anywhere
- * `toSignal()` = ultimate zero-boilerplate -- converts `Observable` to `Signal`, `DestroyRef` handled internally
- * `destroyRef.onDestroy()` = for non-subscription cleanup -- timers, third-party libs, `WebSockets`

The Three Patterns -- When to Use Each:

```
// Pattern 1 -- toSignal -- for template-bound data

// Zero subscription management -- recommended first choice

readonly users = toSignal(this.service.getUsers(), { initialValue: [] as User[] });


// Pattern 2 -- takeUntilDestroyed -- for streams with side effects

// Must set up in ngOnInit or field -- requires explicit destroyRef

private readonly destroyRef = inject(DestroyRef);

this.stream$.pipe(takeUntilDestroyed(this.destroyRef)).subscribe(doSomething);


// Pattern 3 -- destroyRef.onDestroy() -- for non-Observable cleanup

this.destroyRef.onDestroy(() => this.thirdPartyLib.dispose());
```

One-Line Memory Aid: `DestroyRef` is the smart power strip -- plug every subscription in via `takeUntilDestroyed`. `toSignal` plugs itself in automatically. Never implement `ngOnDestroy` just for

subscription cleanup again.

11. Thought-Provoking Question and Answer

DestroyRef can be injected into services, not just components. A providedIn: 'root' service lives for the entire app lifetime -- so DestroyRef in a root service fires only when the app closes. But a component-scoped service has DestroyRef firing when the component is destroyed. How does this difference in DestroyRef lifetime change how you architect services in Angular, and when should a service be component-scoped rather than root-scoped?

When to use component-scoped services (providers: [MyService]):

- * Each component instance needs independent state
- * Service should self-cleanup with its component
- * Two instances of the same component must not share state

When to use root service (providedIn: 'root'):

- * State shared across the entire app
- * HTTP services, API communication
- * Caching that persists across navigation

Practical rule: Default to providedIn: 'root'. Reach for component-scoped when two simultaneous instances of a component should have independent state -- or when a service needs to self-cleanup in sync with a specific component's lifecycle.

12. Related Topics to Learn Next

- * async pipe (Topic 22) -- the template alternative to toSignal -- different trade-offs
- * Angular Signals (Topic 23) -- toSignal() is the bridge between Observables and Signals
- * OnPush Change Detection (Topic 24) -- toSignal() integrates with OnPush automatically
- * RxJS Operators in depth (Topic 46) -- switchMap, mergeMap, forkJoin -- the operators creating subscriptions you manage here

13. Updated Learning Tracker

Topics 1-19: ES6/JS + TypeScript Foundation -- COMPLETE

F1-F5: Angular Foundations Overview -- COMPLETE

Topic 20: RxJS Subscription Management -- COMPLETE

Topic 21: takeUntilDestroyed and DestroyRef -- COMPLETE

Next: Topic 22 -- async Pipe

Topic 22 -- async Pipe in Angular

0. Difficulty and Priority Rating

Difficulty: Beginner-Intermediate -- the pipe itself is one word in a template; understanding null handling, multiple subscription traps, and when to use async pipe vs toSignal takes real examples

Angular Relevance: Critical -- the async pipe eliminates subscription management entirely for template-bound Observables; every Angular app uses it or toSignal -- understanding the trade-offs between them is essential

1. Explanation

The Personal Assistant Analogy

Imagine you are a busy executive. You need information from various departments -- sales figures, inventory reports, customer feedback. Instead of calling each department yourself, waiting on hold, taking notes, and remembering to close each call when you are done -- you hire a personal assistant.

You tell the assistant: "I need the latest sales figures on my desk."

The assistant:

- * Makes the call for you
- * Waits for the response
- * Puts the result on your desk the moment it arrives
- * Updates your desk whenever new figures come in
- * Cancels all open calls automatically when you leave the office

The async pipe is that personal assistant for your Angular template.

You hand it an Observable or Promise and say: "show me whatever this produces." The pipe subscribes for you, puts the value in the template the moment it arrives, updates automatically when new values come, and cancels the subscription when the component is destroyed -- all without you writing a single line of subscription management code.

In Plain English

- * async pipe = a template pipe that subscribes to an Observable or Promise, displays its current value, and unsubscribes automatically when the component is destroyed
- * No .subscribe() needed = the pipe handles the full subscription lifecycle
- * No ngOnDestroy needed = Angular calls .unsubscribe() when the component is destroyed
- * Always shows the latest value = whenever the Observable emits, the template updates automatically
- * Works with both Observables and Promises = same syntax for both

Now the Technical Terms

- * async pipe = Angular's built-in AsyncPipe -- implements PipeTransform and OnDestroy -- subscribes to Observable(T) or Promise(T) and returns T | null
- * Returns null = before the first emission the async pipe returns null -- the template must handle this
- * Automatic subscription = the pipe subscribes when the template is first evaluated
- * Automatic unsubscription = the pipe unsubscribes via its own ngOnDestroy when the component is destroyed

* Change detection = the async pipe triggers change detection when a new value arrives -- works perfectly with OnPush

2. Connection to Prior Concepts

The async pipe is the template-level version of everything learned in Topics 20 and 21:

- * Topic 20 (RxJS Subscription Management) -- the async pipe IS subscription management -- it does it inside the pipe instead of the component class
- * Topic 21 (takeUntilDestroyed and DestroyRef) -- takeUntilDestroyed manages subscriptions in the class; async pipe manages them in the template. Same problem, different layer
- * Topic 18 (Error Handling) -- async pipe has no built-in error handling -- errors from the Observable bubble up uncaught unless you add catchError to the pipe
- * Topic 19 (?. and ??) -- async pipe returns null before the first emission. ?. and ?? in the template are the runtime companions
- * Topic 12 (Modules and Imports) -- async pipe must be imported via AsyncPipe directly in standalone components

Bridging note: "The async pipe moves subscription management from the class into the template. Instead of this.service.getData().subscribe(d => this.data = d) in the class plus ngOnDestroy, you write data\$ | async in the template and the pipe handles everything."

3. Code Example

Example 1 -- The simplest async pipe usage

```
@Component({
  selector: 'app-user-list',
  standalone: true,
  imports: [AsyncPipe],
  template: `
    @for (user of users$ | async ?? []; track user.id) {
      (div){{ user.name }}(/div)
    }
  `
})
export class UserListComponent {

  private readonly userService = inject(UserService);

  // Observable -- NOT subscribed to in the class

  // The async pipe in the template handles everything

  readonly users$ = this.userService.getUsers();
```

```
// No ngOnInit needed

// No ngOnDestroy needed

// No Subscription variable needed

}
```

Example 2 -- Handling null before first emission

```
@Component({
  standalone: true,
  imports: [AsyncPipe],
  template: `
    Option 1 -- @if as -- Angular 17+ recommended
    @if (user$ | async; as user) {
      (h2){{ user.name }}(/h2)
      (p){{ user.email }}(/p)
    } @else {
      (p)Loading...(/p)
    }

    Option 2 -- safe navigation operator + nullish coalescing
    (h2){{ (user$ | async)?.name ?? 'Loading...' }}(/h2)
    `
  })
export class UserProfileComponent {

  private readonly userService = inject(UserService);

  readonly user$ = this.userService.getCurrentUser();

}
```

Example 3 -- Multiple async pipes -- the problem and the solution

```
@Component({
  standalone: true,
  imports: [AsyncPipe],
  template: `
```

PROBLEM: Three separate async pipes = THREE subscriptions = THREE HTTP calls

SOLUTION: @if as -- ONE subscription shared by the block

```
@if (user$ | async; as user) {  
  (h2){{ user.name }}(/h2)  
  (p){{ user.email }}(/p)  
  (span){{ user.role }}(/span)  
}  
`.  
})  
  
export class UserCardComponent {  
  
  private readonly userService = inject(UserService);  
  
  // shareReplay(1) -- replays latest value to any new subscribers  
  readonly user$ = this.userService.getCurrentUser().pipe(  
    shareReplay(1)  
  );  
}
```

Angular Example -- Complete real-world async pipe usage

```
@Component({  
  selector: 'app-dashboard',  
  standalone: true,  
  imports: [AsyncPipe, CurrencyPipe, DatePipe],  
  template: `  
    @if (currentUser$ | async; as user) {  
      (header)  
      (h1)Welcome back, {{ user.name }}(/h1)  
      (span){{ user.role }}(/span)  
    }  
  `})
```

```

@if (ordersState$ | async; as state) {
  @if (state.isLoading) { (div)Loading orders...( /div) }
  @if (state.error) { (div){{ state.error }}( /div) }
  @if (!state.isLoading and !state.error) {
    @for (order of state.orders; track order.id) {
      (div){{ order.id }} | {{ order.total | currency:'GBP' }}( /div)
    } @empty {
      (p)No orders found( /p)
    }
  }
}

(p)Notifications: {{ notifCount$ | async ?? 0 }}( /p)
,
})

export class DashboardComponent {

  private readonly userService = inject(UserService);
  private readonly orderService = inject(OrderService);
  private readonly notifService = inject(NotificationService);

  readonly currentUser$ = this.userService.getCurrentUser();

  readonly ordersState$ = this.orderService.getOrders().pipe(
    map(orders => ({ orders, isLoading: false, error: null as string | null })),
    startWith({ orders: [] as Order[], isLoading: true, error: null }),
    catchError(err => of({
      orders: [] as Order[], isLoading: false,
      error: err?.message ?? 'Failed to load orders'
    })))
  );

  readonly notifCount$ = this.notifService.getCount().pipe(

```

```
catchError(() => of(0))

);

// No ngOnInit, no ngOnDestroy, no Subscription variables

}
```

4. Before and After Refactor

```
// BEFORE -- manual subscription management for every Observable

@Component({
  selector: 'app-user-profile',
  standalone: true,
  imports: [AsyncPipe],
  template: ...
})

export class UserProfileComponent implements OnInit, OnDestroy {

  user: User | null = null;
  recentOrders: Order[] = [];
  notificationCount: number = 0;
  isLoading = true;

  private userSub: Subscription;
  private orderSub: Subscription;
  private notifSub: Subscription;

  private readonly userService = inject(UserService);
  private readonly orderService = inject(OrderService);
  private readonly notifService = inject(NotificationService);

  ngOnInit(): void {

    this.userSub = this.userService.getCurrentUser().subscribe(user => {

      this.user = user; this.isLoading = false;

    });
  }
}
```

```

this.orderSub = this.orderService.getRecentOrders().subscribe(orders => {
  this.recentOrders = orders;
});

this.notifSub = this.notifService.getCount().subscribe(count => {
  this.notificationCount = count;
});
}

ngOnDestroy(): void {
  this.userSub?.unsubscribe();
  this.orderSub?.unsubscribe();
  this.notifSub?.unsubscribe();
}
}

// AFTER -- async pipe -- zero subscription boilerplate

@Component({
  selector: 'app-user-profile',
  standalone: true,
  imports: [AsyncPipe, CurrencyPipe],
  template: `
    @if (user$ | async; as user) {
      (h2){ { user.name } }(/h2)
      (p){ { user.email } }(/p)
    } @else {
      (div)Loading...(/div)
    }

    @for (order of recentOrders$ | async ?? []; track order.id) {
      (div){ { order.id } }(/div)
    }
  `
})

```

```

    (p)Notifications: {{ notificationCount$ | async ?? 0 }}(/p)
    `

  })

  export class UserProfileComponent {

    private readonly userService = inject(UserService);
    private readonly orderService = inject(OrderService);
    private readonly notifService = inject(NotificationService);

    readonly user$ = this.userService.getCurrentUser();
    readonly recentOrders$ = this.orderService.getRecentOrders();
    readonly notificationCount$ = this.notifService.getCount();

    // No ngOnInit, no ngOnDestroy, no Subscription variables
  }

```

5. Common Mistakes and Misconceptions

1. "I can use | async multiple times on the same Observable -- it is the same stream"

Every | async in a template creates an independent subscription. Using user\$ | async three times creates three subscriptions, makes three HTTP calls.

Fix: Use @if (user\$ | async; as user) -- ONE subscription shared by the block.

2. "The async pipe handles errors -- I don't need catchError"

The async pipe has absolutely no error handling. If the Observable errors, the template breaks and the stream is permanently dead. The safety net must be in the Observable chain.

```

// Always add catchError to Observables used with async pipe

readonly users$ = this.userService.getUsers().pipe(

  catchError(err => {

    console.error('Failed to load users:', err);

    return of([] as User[]); // async pipe gets [] instead of an error

  })

);

```

3. "async pipe is always better than toSignal -- keep everything in the template"

async pipe is perfect for display-only data. But it cannot be used in computed() signals, and it returns null before first emission requiring null guards. toSignal is better when you need derived computed values or class-level access.

6. Do's and Don'ts

Use `@if (obs$ | async; as value)` to unwrap once and use multiple times

Do NOT use `| async` on the same Observable multiple times

Always add `catchError` to Observables used with `async` pipe

Do NOT rely on `async` pipe to handle errors -- it crashes the template silently

Use `?? []` or `?? 0` after `| async` for fallback before first emission

Do NOT access properties on `| async` result without null guard

Import `AsyncPipe` directly in standalone components

Do NOT import all of `CommonModule` just to get `AsyncPipe`

Use `shareReplay(1)` when the same Observable is subscribed to multiple times

Do NOT share cold Observables (HTTP calls) without `shareReplay`

Use `toSignal()` when you need the value in `computed()` or class logic

Do NOT use `async` pipe when you need derived computations

7. Debugging Tips

Bug: Template shows nothing -- `async` pipe seems not working

Plain English cause: Observable errored (`async` pipe has no error handling), or `AsyncPipe` not imported in standalone component

Fix: Check browser console for errors. Add `catchError`. Verify `AsyncPipe` is in imports: `[AsyncPipe]`

Bug: `InvalidPipeArgument` for `async`

Plain English cause: You passed the actual value (already unwrapped) to `async` pipe instead of the Observable

Fix: Ensure the field is typed as `Observable(T)` -- `async` pipe needs the Observable itself

Bug: HTTP request fires multiple times

Plain English cause: Multiple `| async` on the same HTTP Observable -- each creates a new subscription

Fix: Use `@if` as pattern to subscribe once, OR add `shareReplay(1)` to the Observable

Bug: `async` pipe shows null instead of data

Plain English cause: `async` pipe returns null before first emission -- no null guard in template

Fix: Add `?? 'fallback'` after the pipe or wrap in `@if` as

Bug: Template not updating when Observable emits

Plain English cause: Observable is not actually emitting a new value -- likely mutation without a new reference

Fix: Verify the Observable actually emits. Check for reference mutation.

8. Real-World Applications

Application 1 -- Paginated Data Table with Loading and Error States

```
@Component({  
  selector: 'app-user-table',
```

```

standalone: true,

imports: [AsyncPipe, CurrencyPipe, DatePipe],

template: `

@if (tableState$ | async; as state) {

@if (state.isLoading) { (div)Loading...(div) }

@if (state.error) {

(div){{ state.error }} (button (click)="onRetry()")Try again(/button)(div)

}

@if (!state.isLoading and !state.error) {

(table)

@for (user of state.users; track user.id) {

(tr)(td){{ user.name }}(/td)(td){{ user.email }}(/td)(tr)

} @empty {

(tr)(td colspan="2")No users found(/td)(tr)

}

(/table)

}

}

`

}))

export class UserTableComponent {

private readonly userService = inject(UserService);

readonly tableState$ = this.userService.getUsers().pipe(
map(result => ({ users: result.items, isLoading: false, error: null as string | null })),
startWith({ users: [] as User[], isLoading: true, error: null }),
catchError(err => of({ users: [] as User[], isLoading: false, error: err?.message ??
'Failed' })))

);

onRetry(): void { window.location.reload(); }

}

```

Application 2 -- Real-time Notification Badge

```
@Component({
  selector: 'app-nav-badge',
  standalone: true,
  imports: [AsyncPipe],
  template: `
    @if (unreadCount$ | async; as count) {
      @if (count > 0) {
        (span){{ count > 99 ? '99+' : count }}(/span)
      }
    }
  `
})

export class NavBadgeComponent {

  private readonly notifService = inject(NotificationService);

  readonly unreadCount$ = this.notifService.getUnreadCount().pipe(
    catchError(() => of(0)),
    distinctUntilChanged()
  );

  // Entire component = 10 lines. Zero subscription management.
}
```

9. Practice Exercises

Exercise 1 -- Beginner

Build a `WeatherComponent` that displays current weather from a `WeatherService.getCurrentWeather()` `Observable`. Use the `async` pipe with `@if` as to show three states: loading, data, and error. Write the error handling in the `Observable` chain -- not in the template. Verify the component has zero subscription management code.

Exercise 2 -- Intermediate

Build a `SearchResultsComponent` that takes a `searchQuery$` `Observable` as an `input()` signal. Implement a search that debounces 300ms, shows loading state, shows results, shows "no results" when empty, and shows an error message if the search fails. The entire subscription must be managed by the `async` pipe.

Exercise 3 -- Advanced

Build a `LiveDashboardComponent` that displays three separate data streams simultaneously -- `statsStream$`, `activityFeed$`, and `alertsStream$`. Each stream should show loading, error with retry, and a "last updated" timestamp. Use ONE async pipe per stream. Add `shareReplay(1)` appropriately. Component class should contain zero subscription management.

10. Key Takeaways

Core Concepts in Plain English:

- * async pipe = the personal assistant -- subscribes for you, delivers values, cancels when you leave
- * Returns null initially = before the first emission -- always guard with `??` fallback or `@if as`
- * One pipe = one subscription -- using `| async` twice on the same Observable = two subscriptions, two HTTP calls
- * `@if as` = subscribe once and use the value multiple times in the block
- * No error handling = errors must be caught with `catchError` in the Observable chain
- * Auto-unsubscribes = pipe calls `.unsubscribe()` in its own lifecycle
- * Works with `OnPush` = pipe triggers change detection on every emission automatically

async pipe vs toSignal vs manual subscribe:

- * async pipe: template only, null before emit, no `computed()`, best for display-only data
- * toSignal: class + template, no null (with `initialValue`), use in `computed()`, best for data + derived logic
- * manual subscribe: class only, you control, error callback, best for side effects

One-Line Memory Aid: async pipe = personal assistant in the template -- subscribes, delivers, cancels when you leave. `@if as` = one subscription shared by the whole block. No error handling built in -- always `catchError` the chain first.

11. Thought-Provoking Question and Answer

Angular now has three ways to consume an Observable in a template -- async pipe, `toSignal()`, and manual subscribe + class property. When would you genuinely choose async pipe over `toSignal`?

When async pipe genuinely wins:

- * Truly display-only data -- no derivation needed -- async pipe is fewer lines
- * Lazy loading -- async pipe subscribes when template renders; `toSignal` subscribes immediately
- * Pre-Angular 16 / no Signals available
- * Quick prototyping

When `toSignal` genuinely wins:

- * Need derived `computed()` values from the data
- * Need value accessible in class methods
- * Building with Signals throughout -- consistency
- * Need `initialValue` to avoid null type handling

Practical rule: Default to `toSignal` in new Signal-based Angular 17+ apps. Reach for async pipe when the component is purely display-focused with no derived logic.

12. Related Topics to Learn Next

- * Angular Signals signal, computed, effect (Topic 23) -- toSignal is the bridge
- * OnPush Change Detection (Topic 24) -- async pipe triggers change detection on emission
- * Component Lifecycle Hooks (Topic 25) -- async pipe has its own lifecycle
- * Angular Reactive Forms (Topic 33) -- valueChanges Observable + async pipe is a common form pattern
- * RxJS Operators in depth (Topic 46) -- startWith, shareReplay, catchError -- operators that make async pipe production-ready

13. Updated Learning Tracker

Topics 1-19: ES6/JS + TypeScript Foundation -- COMPLETE

Topics 20-22: RxJS Subscription Management, takeUntilDestroyed, async Pipe -- COMPLETE

Next: Topic 23 -- Angular Signals

Topic 23 -- Angular Signals (signal, computed, effect)

0. Difficulty and Priority Rating

Difficulty: Intermediate -- the API is small (signal, computed, effect, toSignal) but understanding the reactivity graph, effect scheduling, and when to use Signals vs Observables takes real application experience

Angular Relevance: Critical -- Signals are Angular's modern reactivity system replacing zone.js-driven change detection; every new Angular component should use Signals for state, computed values, and reactive inputs

1. Explanation

The Spreadsheet Analogy

Imagine a spreadsheet with three cells:

- * Cell A1 = 150 (your salary)
- * Cell B1 = 0.2 (your tax rate)
- Cell C1 = A1 B1 (your tax amount -- a formula)

When you change A1 from 150 to 200, Cell C1 automatically updates to 40. You did not tell C1 to recalculate -- the spreadsheet knows C1 depends on A1, so it updates C1 automatically the moment A1 changes. Angular Signals work exactly like this spreadsheet.

- * signal() = Cell A1 -- a value you can read and change
- * computed() = Cell C1 -- a formula that automatically recalculates when its dependencies change
- * effect() = a macro that runs automatically whenever cells it reads change

In Plain English

- * signal(value) = a reactive variable -- when its value changes, everything that reads it is notified automatically
- * computed(fn) = a derived value -- reads from signals, automatically recalculates when any read signal changes, result is cached until dependencies change
- * effect(fn) = a side effect runner -- runs whenever any signal it reads changes, used for logging, syncing with external systems, DOM manipulation
- * Reactivity = signals track who reads them, so when they change, they know exactly what needs to update
- * No subscription needed = unlike Observables, signals do not need .subscribe() -- Angular handles the reactivity graph automatically

Now the Technical Terms

- * signal(T)(initialValue) = creates a WritableSignal(T) -- a reactive container holding a value
- * signal.set(value) = replaces the current value -- triggers all dependents to update
- * signal.update(fn) = computes new value from current -- count.update(c => c + 1) -- immutable update pattern
- * Note: signal.mutate() was removed in Angular 17 -- use update() with spread instead for all object and array mutations
- * computed(fn) = creates a read-only Signal(T) -- Angular memoises the result -- only recomputes when dependencies change
- * effect(fn, options?) = registers a side effect -- runs in the constructor injection context

- * toSignal(observable\$) = converts Observable to Signal -- bridges RxJS and Signals (Topic 21)
- * toObservable(signal) = converts Signal to Observable -- bridge going the other direction

2. Connection to Prior Concepts

Signals sit at the top of the reactivity pyramid, building on everything learned so far:

- * Topic 3 (Closures) -- computed() and effect() are closures that capture their signal dependencies at the moment they run
- * Topic 7 (Pass by Reference) -- signal.update() always returns a new reference for objects and arrays -- this is the immutable update pattern from Topic 7 applied to signals
- * Topic 20 (Subscriptions) -- Signals eliminate manual subscription management for state -- toSignal() bridges the gap for HTTP Observables
- * Topic 21 (takeUntilDestroyed) -- toSignal() uses DestroyRef internally -- the subscription cleanup knowledge from Topic 21 explains why toSignal() works without explicit cleanup
- * Topic 22 (async pipe) -- toSignal() is the class-level alternative -- both solve the Observable-to-template problem, Signals go further by enabling computed() derivation

3. Code Example

Example 1 -- signal() -- the reactive variable

```
import { signal } from '@angular/core';

// CREATE a signal

const count = signal(0); // WritableSignal(number)

const name = signal('Alice'); // WritableSignal(string)

const isLoading = signal(false); // WritableSignal(boolean)

const users = signal([]); // WritableSignal(User[])

// READ a signal -- call it like a function

console.log(count()); // 0

console.log(name()); // 'Alice'

// SET -- replace the value entirely

count.set(5);

console.log(count()); // 5

// UPDATE -- compute new value from current (immutable)

count.update(current => current + 1);

console.log(count()); // 6

// Immutable object update -- always spread to create new reference
```

```

user.update(current => current
? { ...current, name: 'Alice Smith' }
: current
);

// Immutable array updates -- always return new arrays
users.update(current => [...current, newUser]); // add
users.update(current => current.filter(u => u.id !== id)); // remove
users.update(current =>
current.map(u => u.id === id ? { ...u, ...changes } : u) // update one
);

// Note: mutate() was removed in Angular 17
// WRONG: users.mutate(u => u.push(newUser));
// CORRECT: users.update(u => [...u, newUser]);

```

Example 2 -- computed() -- the formula cell

```

import { signal, computed } from '@angular/core';

const price = signal(100);
const quantity = signal(3);
const taxRate = signal(0.2);

// computed -- formula that reads signals
const subtotal = computed(() => price() * quantity());
const tax = computed(() => subtotal() * taxRate());
const total = computed(() => subtotal() + tax());

console.log(subtotal()); // 300
console.log(total()); // 360

// When a dependency changes -- all dependents update
price.set(150);
console.log(subtotal()); // 450

```



```

console.log(total()); // 540

// computed is MEMOISED -- only recomputes when dependencies change
console.log(total()); // 540 -- cached result, no recomputation

// computed with arrays and filtering
const users = signal([
  { id: 1, name: 'Alice', role: 'admin', isActive: true },
  { id: 2, name: 'Bob', role: 'user', isActive: false },
  { id: 3, name: 'Carol', role: 'user', isActive: true }
]);

const searchTerm = signal('');
const filterRole = signal('all');

const filteredUsers = computed(() => {
  const term = searchTerm().toLowerCase();
  const role = filterRole();
  return users().filter(user => {
    const matchesTerm = !term || user.name.toLowerCase().includes(term);
    const matchesRole = role === 'all' || user.role === role;
    return matchesTerm and matchesRole and user.isActive;
  });
});

```

Example 3 -- effect() -- the side effect runner

```

import { signal, computed, effect } from '@angular/core';

const theme = signal('light');
const userId = signal(null);
const isLoggedIn = computed(() => userId() !== null);

// effect -- runs immediately, re-runs when dependencies change

// Side effect 1: sync theme to DOM

```

```

effect(() => {

  document.body.setAttribute('data-theme', theme());

  console.log('Theme changed to:', theme());

});

theme.set('dark'); // effect re-runs automatically

// Side effect 2: localStorage sync
effect(() => {

  localStorage.setItem('theme', theme());

});

// Writing to signals INSIDE effect -- needs allowSignalWrites
const lastTheme = signal('');

// With allowSignalWrites option
effect(() => {

  lastTheme.set(theme());

}, { allowSignalWrites: true });

// Better pattern: use computed instead when possible
const lastTheme2 = computed(() => theme()); // no effect needed

```

Angular Example -- Signals in a real component

```

import {

  Component, signal, computed, effect, inject

} from '@angular/core';

import { toSignal } from '@angular/core/rxjs-interop';

import { FormsModule } from '@angular/forms';

@Component({

  selector: 'app-product-catalogue',

  standalone: true,

  imports: [FormsModule],

```

```

template: `


```

```

readonly filteredProducts = computed(() => {
  const term = this.searchTerm().toLowerCase();
  return this.products().filter(p =>
    !term || p.name.toLowerCase().includes(term)
  );
});

constructor() {
  effect(() => {
    const product = this.selectedProduct();
    if (product) {
      this.analytics.track('product_viewed', { productId: product.id });
    }
  });
}

selectProduct(product: Product): void {
  this.selectedProduct.set(product);
}
}

```

4. Before and After Refactor

```

// BEFORE -- BehaviorSubject-based state

@Component({ selector: 'app-cart', standalone: true, template: '...' })
export class CartComponent implements OnInit, OnDestroy {

  private readonly cartService = inject(CartService);

  private readonly itemsSubject = new BehaviorSubject([]);
  private readonly isLoadingSubject = new BehaviorSubject(false);
  private readonly errorSubject = new BehaviorSubject(null);

```

```

readonly items$ = this.itemsSubject.asObservable();

readonly isLoading$ = this.isLoadingSubject.asObservable();

readonly error$ = this.errorSubject.asObservable();


readonly itemCount$ = this.items$.pipe(map(items => items.length));

readonly total$ = this.items$.pipe(
  map(items => items.reduce((sum, i) => sum + i.price * i.qty, 0))
);


private readonly destroy$ = new Subject();


ngOnInit(): void {

  this.isLoadingSubject.next(true);

  this.cartService.getCart().pipe(
    takeUntil(this.destroy$)
  ).subscribe({

    next: items => { this.itemsSubject.next(items); this.isLoadingSubject.next(false); },
    error: err => { this.errorSubject.next(err.message); this.isLoadingSubject.next(false); }
  });
}


ngOnDestroy(): void { this.destroy$.next(); this.destroy$.complete(); }

// 60+ lines -- 3 subjects, 7 derived observables, ngOnDestroy
}


// AFTER -- Signal-based state

@Component({ selector: 'app-cart', standalone: true, template: '...' })
export class CartComponent {

  private readonly cartService = inject(CartService);

  readonly items = signal([]);

  readonly isLoading = signal(false);

```

```

readonly error = signal(null);

readonly itemCount = computed(() => this.items().length);

readonly total = computed(() =>
  this.items().reduce((sum, i) => sum + i.price * i.qty, 0)
);

readonly isEmpty = computed(() => this.items().length === 0);

constructor() {
  const cartData = toSignal(
    this.cartService.getCart().pipe(
      catchError(err => { this.error.set(err.message); return of([]); })
    ),
    { initialValue: [] }
  );

  effect(() => { this.items.set(cartData()); }, { allowSignalWrites: true });
}

addItem(item: CartItem): void {
  this.items.update(current => [...current, item]);
}

removeItem(id: number): void {
  this.items.update(current => current.filter(i => i.id !== id));
}

// No ngOnInit, no ngOnDestroy, no destroy$, no subjects
}

```

5. Common Mistakes and Misconceptions

1. "I can write to signals inside computed() -- it's just a function"

computed() must be a pure function -- no side effects, no signal writes. Writing to a signal inside computed() creates circular dependencies and causes Angular to throw an error.

```
// WRONG: Writing to a signal inside computed
```

```

const result = computed(() => {

  doubled.set(count() * 2); // throws Error: computed cannot write to signal

  return count() * 2;

});

// CORRECT: computed -- read-only derived value

const result = computed(() => count() * 2);

```

2. "effect() is like ngOnChanges -- it replaces lifecycle hooks"

effect() is for side effects on the outside world -- DOM manipulation, localStorage, analytics, logging. It is NOT for computing derived state (use computed()) and NOT for handling lifecycle events.

```

// WRONG: Using effect for derived state

effect(() => {

  total.set(this.items().reduce((s, i) => s + i.price, 0));

}, { allowSignalWrites: true });

// CORRECT: computed for derived state

readonly total = computed(() =>

  this.items().reduce((s, i) => s + i.price, 0)

);

// CORRECT: effect for genuine side effects only

effect(() => {

  localStorage.setItem('cart-count', String(this.total()));

});

```

3. "Signals replace Observables -- I can use signals for everything"

Signals are synchronous reactive state containers. Observables handle async event streams, HTTP, WebSockets, complex operators, and time-based logic. They are complementary, not competing.

```

// CORRECT: toSignal bridges Observable -> Signal cleanly

readonly users = toSignal(

  this.http.get('/api/users'),

  { initialValue: [] }

);

// HTTP stays Observable -- template works with Signal

```

6. Do's and Don'ts

Use computed() for ALL derived state

Do NOT use effect() to compute derived values

Always use update() with spread for immutable updates

Do NOT use mutate() -- it was removed in Angular 17

Use effect() only for side effects on the outside world

Do NOT write to signals inside computed()

Call effect() in the constructor (injection context)

Do NOT call effect() in ngOnInit or other lifecycle hooks

Use toSignal() to bridge HTTP Observables into Signal-based components

Do NOT try to replace Observables entirely

Use signal.asReadOnly() for publicly exposed signals in services

Do NOT expose WritableSignal publicly from services

7. Debugging Tips

Bug: Error -- A computed value cannot write to a signal

Plain English cause: A set() or update() was called inside a computed() -- computed must be a pure function

Fix: Move the signal write to an effect() with { allowSignalWrites: true }, or restructure so the derived value uses computed() without writing

Bug: Error -- effect() can only be used within an injection context

Plain English cause: effect() was called in ngOnInit or another method -- not in a constructor or field initialiser

Fix: Move effect() calls to the constructor

Bug: Template not updating when signal changes

Plain English cause: The signal value was mutated directly instead of using .set() or .update()

Fix: Always use .set() or .update() -- never mutate the value directly

```
// WRONG: Direct mutation

this.users().push(newUser); // mutating in place -- template won't update

// CORRECT: Immutable update

this.users.update(u => [...u, newUser]); // new array -- template updates
```

Bug: toSignal throws injection context error

Plain English cause: toSignal() was called inside ngOnInit -- it uses inject() internally

Fix: Call toSignal() as a class field initialiser -- not inside a lifecycle hook

8. Real-World Applications

Application 1 -- Shopping Cart with Signal-Based State


```

@Injectables({ providedIn: 'root' })

export class CartService {

  private readonly _items = signal([]);

  private readonly _voucher = signal(null);

  readonly items = this._items.asReadonly();

  readonly voucher = this._voucher.asReadonly();

  readonly itemCount = computed(() => this._items().length);

  readonly subtotal = computed(() =>
    this._items().reduce((sum, item) => sum + item.price * item.qty, 0)
  );

  readonly discount = computed(() => {
    const code = this._voucher();
    return code === 'SAVE10' ? this.subtotal() * 0.1 : 0;
  });

  readonly total = computed(() => this.subtotal() - this.discount());

  readonly isEmpty = computed(() => this._items().length === 0);

  addItem(product: Product): void {
    this._items.update(items => {
      const existing = items.find(i => i.id === product.id);

      if (existing) {
        return items.map(i =>
          i.id === product.id ? { ...i, qty: i.qty + 1 } : i
        );
      }

      return [...items, { ...product, qty: 1 }];
    });
  }

  removeItem(id: number): void {

```

```

    this._items.update(items => items.filter(i => i.id !== id));
  }

  clearCart(): void { this._items.set([]); }
}

```

Application 2 -- Signal-Based Search and Filter

```

@Component({
  selector: 'app-catalogue',
  standalone: true,
  imports: [FormsModule],
  template: `
    (input [ngModel]="search()" (ngModelChange)="search.set($event)" placeholder="Search...")

    (p){ { results().length } } results

    @if (isFiltered()) { (span)(filtered from { { allProducts().length } })(/span) }

    (/p)

    @for (product of results(); track product.id) {
      (div){ { product.name } }(/div)
    } @empty {
      (p)No products match your filters(/p)
    }
  `
})
export class CatalogueComponent {

  private readonly productService = inject(ProductService);

  readonly allProducts = toSignal(
    this.productService.getAll(),
    { initialValue: [] }
  );
}

```

```

readonly search = signal('');

readonly category = signal('all');

readonly results = computed(() => {
  const term = this.search().toLowerCase();

  const cat = this.category();

  return this.allProducts()
    .filter(p =>
      (!term || p.name.toLowerCase().includes(term)) and
      (cat === 'all' || p.category === cat)
    );
});

readonly isFiltered = computed(() =>
  this.search() !== '' || this.category() !== 'all'
);
}

```

9. Practice Exercises

Exercise 1 -- Beginner

Build a CounterComponent with three signals: count, step (default 1), and maxCount (default 10). Add computed() values: doubled, isAtMax, percentComplete. Add buttons to increment by step, decrement by step, reset to 0, and change the step value. Add an effect() that logs every time count changes.

Exercise 2 -- Intermediate

Build a TaskListComponent managing tasks (id, title, priority, isComplete, tags[]). Use signals and computed for: all tasks, filtered tasks (by status), tasks grouped by priority, completion percentage. Implement add, complete, delete, and filter operations -- all using immutable update patterns. Use toSignal() for the initial task load from a service.

Exercise 3 -- Advanced

Build a signal-based FormStateService (component-scoped) that manages a multi-step form with five steps. Each step has its own signal for form values. Computed signals derive: current step validity, overall form validity, completed steps array, and can-proceed flag. An effect syncs form progress to localStorage. A second effect sends analytics when each step is completed.

10. Key Takeaways

Core Concepts in Plain English:

- * signal(v) = reactive variable -- .set() replaces, .update(fn) derives from current

- * `computed(fn)` = formula cell -- read-only, memoised, auto-updates when dependencies change
- * `effect(fn)` = side effect runner -- DOM, localStorage, analytics only -- runs in constructor
- * Read with `()` -- always call signals like functions: `count()` not `count`
- * Always immutable -- spread to create new references: `signal.update(s => [...s, item])`
- * `signal.mutate()` is removed in Angular 17 -- always use `update()` with spread
- * No subscription needed -- Angular manages the reactivity graph automatically
- * `toSignal(obs$, { initialValue })` -- bridge Observable to Signal for HTTP data

Quick Reference:

```
// Create

const count = signal(0);

// Read

count() // always call with ()

// Write

count.set(5); // replace value

count.update(c => c + 1); // derive from current

users.update(u => [...u, newUser]); // immutable array add

users.update(u => u.filter(x => x.id !== id)); // immutable remove

// Derive

const doubled = computed(() => count() * 2); // read-only, memoised

// Side effect

effect(() => { localStorage.setItem('c', String(count())); }); // constructor only

// Bridge

const users = toSignal(this.svc.getUsers(), { initialValue: [] });
```

One-Line Memory Aid: `signal()` = reactive spreadsheet cell. `computed()` = formula. `effect()` = macro for the outside world only. Read with `()`. Update with spread. No subscriptions, no cleanup.

11. Thought-Provoking Question and Answer

Angular 17+ introduced `input()` and `output()` functions as signal-based alternatives to decorators. If component inputs are now Signals, the entire component becomes a reactive graph. What does this signal-based component tree mean for `OnPush` change detection, and does it make `OnPush` obsolete?

Signal inputs make the component tree reactive without manual change detection hints. What happens when `updateEmail()` runs:

1. currentUser signal changes
2. user input signal in child receives new value
3. userName computed: name unchanged -- NOT recomputed
4. userEmail computed: email changed -- recomputed
5. Template: only the email line updates -- not the name line. Surgical precision -- impossible with traditional @Input() + OnPush.

Does signals make OnPush obsolete? Not yet -- but it changes the reason you would use it.

Signal-based components automatically get fine-grained updates. OnPush still benefits non-signal parts of the tree. Together: signals tell Angular exactly what changed; OnPush prevents unnecessary tree traversal.

Practical rule: Use both together today: signals for fine-grained reactivity + OnPush to prevent unnecessary parent-triggered checks.

12. Related Topics to Learn Next

- * OnPush Change Detection (Topic 24) -- signals + OnPush = Angular's performance story
- * Component Lifecycle Hooks (Topic 25) -- how signals interact with Angular's lifecycle
- * input() and output() signal functions (Topic 26) -- the signal-based component communication API
- * Services and Dependency Injection (Topic 27) -- signal-based services
- * RxJS Operators in depth (Topic 46) -- toObservable(), toSignal() -- the bridges between Signals and RxJS

13. Updated Learning Tracker

Topics 1-19: ES6/JS + TypeScript Foundation -- COMPLETE

Topics 20-23: RxJS, DestroyRef, async Pipe, Angular Signals -- COMPLETE

Next: Topic 24 -- OnPush Change Detection

Topic 24 -- OnPush Change Detection

0. Difficulty and Priority Rating

Difficulty: Intermediate -- adding OnPush is one line; understanding why templates stop updating (and how to fix them without removing OnPush) requires understanding immutability, Signals, and Angular's change detection model

Angular Relevance: Critical -- OnPush is the difference between an Angular app that scales and one that slows down as it grows; combined with Signals it makes your code zoneless-ready today

1. Explanation

The Security Guard Analogy

Imagine a large office building. Every evening, a security guard does rounds -- checking every single room to see if anything has changed. Even rooms where nobody has been all day. Even the archive room that has not been opened in months. Every room, every evening, regardless.

That is Angular's default change detection -- check every component, every time anything happens.

Now imagine a smarter security system. Instead of checking every room, each room has a motion sensor. The guard only checks rooms where the sensor was triggered.

OnPush is that smart security system. Angular only checks a component when specific, known triggers fire -- a new `input()` value, an event from the component itself, an async pipe emitting, or a Signal changing.

- * Default: every room checked every evening -- wasteful but safe
- * OnPush: only rooms where sensor triggered -- efficient and smart

In Plain English

- * Change detection = Angular's process of checking if component templates need to be re-rendered
- * Default = Angular checks every component in the tree on every change detection cycle
- * OnPush = Angular only checks a component when it has a specific reason to
- * The four triggers = what tells Angular "check this OnPush component now"
- * Immutability connection = OnPush checks if `input()` values changed by reference -- mutating an object's contents without creating a new reference means OnPush never notices the change
- * Signals are OnPush-native = Signal changes notify Angular's reactivity graph directly

Now the Technical Terms

- * Change detection cycle = the process Angular runs to compare current state with previous state and update the DOM
- * `ChangeDetectionStrategy.Default` = checks the entire component tree on every event, timer, HTTP response, and Promise resolution
- * `ChangeDetectionStrategy.OnPush` = marks a component's view as dormant -- Angular only re-checks it when specific triggers fire
- * `markForCheck()` = manually tells Angular "check this OnPush component on the next cycle" -- a manual fallback, not the primary tool
- * `detectChanges()` = immediately runs change detection for this component and its children -- synchronous, use sparingly

* Zoneless Angular = Angular 18+ mode without zone.js -- Signals replace zone.js as the notification mechanism

2. Connection to Prior Concepts

OnPush is the reason every previous topic was taught the way it was:

- * Topic 7 (Pass by Reference) -- OnPush checks if input() values changed by reference. Mutating an object (this.user.name = 'Bob') does not change the reference -- OnPush sees the same key and does not check. Creating a new object ({ ...this.user, name: 'Bob' }) changes the reference -- OnPush notices
- * Topic 14 (readonly and Immutability) -- readonly arrays and Readonly(T) prevent the accidental mutations that break OnPush
- * Topic 22 (async pipe) -- the async pipe triggers change detection internally every time the Observable emits
- * Topic 23 (Signals) -- Signals notify Angular's change detection directly. A Signal change in an OnPush component automatically marks it for checking
- * Topics 20/21 (Subscriptions) -- async pipe and toSignal() handle OnPush notification automatically; manual subscriptions updating plain class properties do not

Bridging note: "OnPush is the reason the previous topics were structured around immutability, Signals, and the async pipe. They are not independent ideas -- they are pieces of the same performance puzzle. OnPush is the rule; immutability, Signals, and async pipe are how you follow the rule without fighting it."

3. Code Example

Example 1 -- Default vs OnPush -- seeing the difference

```
// DEFAULT change detection -- checks on everything

@Component({
  selector: 'app-default',
  standalone: true,
  template: (p){ { user().name } }(/p)
})

export class DefaultComponent {
  user = input();

  // Checked on EVERY change detection cycle:

  // - Any click anywhere in the app

  // - Any setTimeout or setInterval firing

  // - Any HTTP response completing
}
```

```
// ONPUSH -- only checks when triggered

@Component({
  selector: 'app-onpush',
  changeDetection: ChangeDetectionStrategy.OnPush,
  standalone: true,
  template: (p){{ user().name }}(/p)
})

export class OnPushComponent {
  user = input();

  // Only checked when:

  // 1. user input() value reference changes
  // 2. An event fires from THIS component
  // 3. An async pipe in this template emits
  // 4. A Signal read in this template changes
  // 5. markForCheck() or detectChanges() called manually (fallback)
}
```

Example 2 -- The four OnPush triggers

```
@Component({
  selector: 'app-triggers-demo',
  changeDetection: ChangeDetectionStrategy.OnPush,
  standalone: true,
  imports: [AsyncPipe],
  template: `
    (p){{ notifications$ | async | json }}(/p) // Trigger 2: async pipe
    (p){{ count() }}(/p) // Trigger 4: Signal
    (button (click)="onButtonClick()")Click me(/button) // Trigger 3: component event
    (p){{ user()?.name }}(/p) // Trigger 1: input
  `
})

export class TriggersDemoComponent {
```



```

user = input(); // Trigger 1

readonly count = signal(0); // Trigger 4

private readonly notifService = inject(NotificationService);

readonly notifications$ = this.notifService.getNotifications(); // Trigger 2

onButtonClick(): void {

  this.count.update(c => c + 1); // Trigger 3 + 4

}

}

```

Example 3 -- The immutability requirement in action

```

@Component({
  selector: 'app-parent',
  standalone: true,
  imports: [UserCardComponent],
  template: (app-user-card [user]="currentUser()" /)
})

export class ParentComponent {

  currentUser = signal({ id: 1, name: 'Alice', role: 'user' });

  // WRONG: Mutation -- OnPush child WON'T update

  updateNameWrong(): void {

    const user = this.currentUser();

    user.name = 'Bob'; // mutates existing object -- same reference

    // OnPush child: "same reference -- skip check"

  }

  // CORRECT: New reference -- OnPush child WILL update

  updateNameCorrect(): void {

    this.currentUser.update(u => ({ ...u, name: 'Bob' })); // new object

    // OnPush child: "new reference -- check this component"

  }
}

```

```

}

// Arrays -- same rule applies

// WRONG: push mutates -- same reference
addUserWrong(user: User): void {
  this.users().push(user); // mutates array inside signal
}

// CORRECT: update with spread -- new reference
addUserCorrect(user: User): void {
  this.users.update(u => [...u, user]); // new array
}

```

Angular Example -- A complete OnPush component done right

```

@Component({
  selector: 'app-order-dashboard',
  changeDetection: ChangeDetectionStrategy.OnPush,
  standalone: true,
  imports: [AsyncPipe, CurrencyPipe, DatePipe],
  template: `
    (header)

    (h1)Orders -- {{ pendingCount() }} pending(/h1)

    (span [class.alert]="hasUrgentOrders()")

    {{ hasUrgentOrders() ? 'URGENT ORDERS' : 'All clear' }}

    (/span)

    (/header)

    @if (currentUser$ | async; as user) {
      (p)Logged in as: {{ user.name }}(/p)
    }

    @for (order of filteredOrders(); track order.id) {
      (div (click)="selectOrder(order)")
    }
  `
})

```

```

    {{ order.id }} | {{ order.total | currency:'GBP' }}

  (/div)

  } @empty {

    (p)No orders match the current filter(/p)

  }
  `

  })

  export class OrderDashboardComponent {

    private readonly orderService = inject(OrderService);
    private readonly userService = inject(UserService);

    private readonly allOrders = toSignal(
      this.orderService.getOrders(),
      { initialValue: [] }
    );

    readonly currentUser$ = this.userService.getCurrentUser();

    readonly activeFilter = signal('all');
    readonly selectedOrder = signal(null);

    readonly filteredOrders = computed(() => {
      const filter = this.activeFilter();
      return this.allOrders().filter(o =>
        filter === 'all' || o.status === filter
      );
    });

    readonly pendingCount = computed(() =>
      this.allOrders().filter(o => o.status === 'pending').length
    );

    readonly hasUrgentOrders = computed(() =>

```

```

    this.allOrders().some(o => o.isUrgent and o.status === 'pending')
  );

  selectOrder(order: Order): void {
    this.selectedOrder.set(order); // Signal change -- OnPush notified
  }
}

```

4. Before and After Refactor

```

// BEFORE -- no OnPush, manual subscriptions, performance drain

@Component({
  selector: 'app-product-list',
  standalone: true,
  imports: [CurrencyPipe],
  template: ...
})

export class ProductListComponent implements OnInit, OnDestroy {

  products: Product[] = [];
  displayProducts: Product[] = [];

  private readonly productService = inject(ProductService);
  private readonly destroyRef = inject(DestroyRef);

  ngOnInit(): void {
    this.productService.getProducts().pipe(
      takeUntilDestroyed(this.destroyRef)
    ).subscribe(products => {
      this.products = products;
      this.displayProducts = products;
    });
  }
}

```

```

}

// AFTER -- OnPush + Signals + immutability -- production ready

@Component({
  selector: 'app-product-list',
  changeDetection: ChangeDetectionStrategy.OnPush,
  standalone: true,
  imports: [CurrencyPipe],
  template: `
    (input [value]="searchTerm()" (input)="onSearch($event)")

    @for (product of displayProducts(); track product.id) {
      (div){{ product.name }} -- {{ product.price | currency:'GBP' }}{/div}
    }

    (p){{ displayProducts().length }} products{/p}
  `
})

export class ProductListComponent {

  private readonly productService = inject(ProductService);

  private readonly allProducts = toSignal(
    this.productService.getProducts(),
    { initialValue: [] }
  );

  readonly searchTerm = signal('');

  readonly displayProducts = computed(() => {
    const term = this.searchTerm().toLowerCase();

    return !term
      ? this.allProducts()

```

```

    : this.allProducts().filter(p => p.name.toLowerCase().includes(term));

  });

  onSearch(event: Event): void {

    this.searchTerm.set((event.target as HTMLInputElement).value);

  }

  // No ngOnInit, no ngOnDestroy, no Subscription, no markForCheck

}

```

5. Common Mistakes and Misconceptions

1. "OnPush means the component never updates -- it is broken"

When developers first add OnPush and find their component not updating, the instinct is to remove it. The component is working exactly as designed -- it is not updating because the data is being mutated without new references. The fix is never to remove OnPush -- it is to fix the data update pattern.

```

// The correct diagnosis:

// OnPush not updating -) ask "am I creating new references?"

// If no -) fix the data mutation, keep OnPush

// NEVER -) remove OnPush because it is "causing problems"

```

2. "OnPush only matters for big apps -- do not bother for small ones"

OnPush is a habit and an architecture decision -- not a scaling feature you add later. A component that works correctly with OnPush works because its data flow is correct -- immutable updates, Signals, proper event handling. Skipping OnPush during development builds mutation-heavy patterns that are very difficult to refactor later.

3. "Manual subscribe in an OnPush component with this.data = result works fine"

Manual subscribe updating a class property in an OnPush component will often appear to work during development but silently fails in specific scenarios. Without Signals, async pipe, or manual markForCheck(), a subscription callback has no way to notify OnPush.

```

// Fix 1 -- use toSignal (preferred)

readonly users = toSignal(this.userService.getUsers(), { initialValue: [] });

// Fix 2 -- use async pipe (preferred)

readonly users$ = this.userService.getUsers();

// Template: @for (user of users$ | async ?? []); track user.id

// Fix 3 -- markForCheck as fallback

```

```

private readonly cdr = inject(ChangeDetectorRef);

ngOnInit(): void {

  this.userService.getUsers().pipe(

    takeUntilDestroyed(this.destroyRef)

  ).subscribe(users => {

    this.users = users;

    this.cdr.markForCheck();

  });

}

```

6. Do's and Don'ts

Add ChangeDetectionStrategy.OnPush to every component -- make it the default habit

Do NOT leave components on Default and plan to add OnPush later

Use immutable update patterns -- spread, map, filter -- never push, splice in-place

Do NOT mutate input() values or arrays

Use Signals for component state -- Angular notifies OnPush automatically

Do NOT use manual subscribe + class property in OnPush without markForCheck()

Use async pipe for Observable data -- change detection handled internally

Do NOT use manual subscribe in OnPush templates expecting auto-update

Reach for markForCheck() only when neither Signals nor async pipe is feasible

Do NOT call detectChanges() habitually -- it is synchronous and heavier

Ensure parent components are also OnPush

Do NOT add OnPush only to leaf components -- whole tree must use it for maximum benefit

7. Debugging Tips

Bug: Template not updating after data changes in OnPush component

Plain English cause: Data mutated without creating a new reference -- OnPush sees the same key and skips checking

Diagnosis: Check if before === after reference -- if true, you mutated in place

Fix: Use spread to create new references -- { ...obj } for objects, [...arr] for arrays

Bug: async pipe data showing but Signal data not updating in OnPush

Plain English cause: A plain class property was used instead of an actual Signal

Fix: Ensure the property is declared with signal() -- plain properties provide no notification

Bug: OnPush component updates inconsistently -- sometimes yes, sometimes no

Plain English cause: Click events trigger change detection for the subtree (Trigger 3). HTTP callbacks do not

Fix: Use async pipe or toSignal() for HTTP data

Bug: Adding markForCheck() works but feels wrong

Plain English cause: markForCheck() is correct as a fallback but signals a missed opportunity

Guidance: Use markForCheck() as a temporary fix. Refactor to async pipe or toSignal() for the clean solution.

8. Real-World Applications

Application 1 -- Large Data Table with Frequent Updates

```
@Component({
  selector: 'app-orders-table',
  changeDetection: ChangeDetectionStrategy.OnPush,
  standalone: true,
  imports: [CurrencyPipe, DatePipe, OrderRowComponent],
  template: `
    @for (order of orders(); track order.id) {
      (app-order-row [order]="order" (statusChanged)="onStatusChange($event)" /)
    }
    (p)Total: {{ totalValue() | currency:'GBP' }}(/p)
  `
})
export class OrdersTableComponent {

  private readonly orderService = inject(OrderService);

  private readonly ordersSignal = toSignal(
    timer(0, 30_000).pipe(
      switchMap(() => this.orderService.getOrders()),
      catchError(() => of([]))
    ),
    { initialValue: [] }
  );

  readonly orders = computed(() => this.ordersSignal());
  readonly totalValue = computed(() =>
    this.orders().reduce((sum, o) => sum + o.total, 0)
  );
}
```



```

);

onStatusChange(event: { id: number; status: string }): void {
  this.orderService.updateStatus(event.id, event.status);
}
}

@Component({
  selector: 'app-order-row',
  changeDetection: ChangeDetectionStrategy.OnPush,
  standalone: true,
  imports: [CurrencyPipe, DatePipe],
  template: `
    (td){ { order().id } }(/td)
    (td){ { order().total | currency:'GBP' } }(/td)
    (td){ { order().status } }(/td)
    (td)(button (click)="onComplete()")Complete(/button)(/td)
  `
})

export class OrderRowComponent {
  order = input.required();
  statusChanged = output();

  onComplete(): void {
    this.statusChanged.emit({ id: this.order().id, status: 'completed' });
  }
}

// 1000 rows -- only rows whose order input has new reference get re-rendered

```

Application 2 -- Notification Badge Tree

```

// When a new notification arrives:

// AppShell: currentUserId unchanged -) NOT checked

// NavBar: userId input unchanged -) NOT checked

```

```
// Badge: count$ async pipe emits -) CHECKED

// Only 1 component re-rendered out of the whole tree
```

9. Practice Exercises

Exercise 1 -- Beginner

Take a component with `ChangeDetectionStrategy.Default` that has a `users: User[]` property loaded via `subscribe`, an `addUser()` method using `push`, and a `removeUser()` method using `splice`. Add `ChangeDetectionStrategy.OnPush`. Observe what breaks. Fix every broken part using: `toSignal()` for the HTTP call, `signal()` for the array, and `update()` with `spread/filter` for mutations. Document each fix.

Exercise 2 -- Intermediate

Build a `ProductGridComponent` with `OnPush` that receives a category via `input()`, loads products for that category via HTTP using `toSignal()`, allows inline editing of product names, filters products by a local search signal, and shows a count of filtered results. Add `console.log` in `ngDoCheck()` -- it should log minimally.

Exercise 3 -- Advanced

Build a `LiveDataDashboard` component tree with four levels. Apply `OnPush` to all four levels. The dashboard polls an API every 10 seconds for metrics data. Use Signals throughout. Add `ngDoCheck()` with a counter to every component and verify: `MetricValueComponent` logs only when its specific metric value changes, not on every poll cycle for unchanged metrics.

10. Key Takeaways

Core Concepts in Plain English:

- * Change detection = Angular checking if templates need to update -- runs on every async event by default
- * Default = checks every component on every event anywhere in the app
- * `OnPush` = checks a component only when specific triggers fire
- * The core rule = `OnPush` checks `input()` values by reference -- mutation without new reference = invisible to `OnPush`
- * Immutability is required = `spread` for objects, `filter/map` for arrays -- always new references
- * Signals automatically notify = no `markForCheck()` needed -- Signals are `OnPush`-native
- * `async pipe` automatically notifies = change detection handled internally

The Four `OnPush` Triggers:

1. `input()` value gets new reference -- parent passes new object/array reference
2. Event from this component -- (click), (input), (submit) etc.
3. `async pipe` emits -- handles notification internally
4. Signal changes -- Angular's reactive graph notified directly

Plus manual fallbacks:

5. `markForCheck()` -- schedule check for next cycle
6. `detectChanges()` -- check immediately, synchronously (use sparingly)

One-Line Memory Aid: `OnPush` is the motion sensor -- only lights up when specific triggers fire. New `input()` reference, component event, `async pipe` emission, or Signal change. Mutation without new reference = walking invisibly past the sensor.

11. Thought-Provoking Question and Answer

Angular's team has announced "zoneless Angular" -- running Angular without zone.js entirely. If zone.js is removed, what replaces it, and how does understanding OnPush help you write code that works in both zoned and zoneless Angular today?

Zone.js is the PA system that announces to every room when something happens. Zoneless Angular removes the PA system -- each room raises its own hand when it needs attention. Signals are the hand-raising mechanism.

The three patterns that work in both zoned and zoneless:

```
// PATTERN 1 -- Signals (zoneless native -- best)

@Component({
  changeDetection: ChangeDetectionStrategy.OnPush,
  standalone: true,
  template: {{ count() }}
})

export class ZonelessReadyComponent {
  readonly count = signal(0);
  increment(): void { this.count.update(c => c + 1); }
}

// PATTERN 2 -- async pipe (works in both)

@Component({
  changeDetection: ChangeDetectionStrategy.OnPush,
  standalone: true,
  imports: [AsyncPipe],
  template: {{ data$ | async }}
})

export class ZonelessReadyComponent {
  private readonly service = inject(DataService);
  readonly data$ = this.service.getData();
}

// PATTERN 3 -- toSignal (best for HTTP data)

@Component({
  changeDetection: ChangeDetectionStrategy.OnPush,
```

```
standalone: true,  
  
template: {{ data() }}  
  
})  
  
export class ZonelessReadyComponent {  
  
  private readonly service = inject(DataService);  
  
  readonly data = toSignal(this.service.getData(), { initialValue: null });  
  
}
```

Practical rule: Write OnPush + Signals + async pipe today and your code is already zoneless-ready.

12. Related Topics to Learn Next

- * Component Lifecycle Hooks (Topic 25) -- ngDoCheck is where you can observe change detection cycles
- * input() and output() signal functions (Topic 26) -- signal inputs + OnPush = the perfect combination
- * Services and Dependency Injection (Topic 27) -- service-level signals shared across OnPush components
- Angular Template Syntax (Topic 28) -- @for with track replaces ngFor with trackBy
- * Angular Reactive Forms (Topic 33) -- forms + OnPush requires careful handling

13. Updated Learning Tracker

Topics 1-24: All through OnPush Change Detection -- COMPLETE

Next: Topic 25 -- Component Lifecycle Hooks

Topic 25 -- Component Lifecycle Hooks

0. Difficulty and Priority Rating

Difficulty: Beginner-Intermediate -- the individual hooks are simple; knowing which hook to use for which task, and how they interact with Signals, OnPush, and input() signals takes real practice

Angular Relevance: Critical -- every Angular component goes through a predictable lifecycle; knowing when each hook fires is what separates components that work correctly from ones that have subtle timing bugs

1. Explanation

The Employee Onboarding Analogy

Imagine a new employee joining a company. Their working life follows a predictable sequence:

- * constructor -- the paperwork is signed, the employee exists, but has not started work yet. Do not ask them to do anything complex yet -- their desk is not set up. Inject services here only.
- * ngOnChanges -- they receive their job description and responsibilities (the input() values). Happens before their first day and every time their role changes
- * ngOnInit -- they are at their desk, settled in, ready to do real work. Load data, set up subscriptions, start processes
- * (change detection cycles) -- Angular checks their work regularly
- * ngAfterViewInit -- the building is fully constructed, they can now see and use all the rooms (child components, DOM elements)
- * ngOnDestroy -- final cleanup, hand over work, close accounts

In Plain English

- * Lifecycle hooks = methods Angular calls at specific moments in a component's life -- from creation to destruction
- * constructor = called when Angular creates the class instance -- before inputs arrive -- for inject() calls only
- * ngOnChanges = called when input() values change -- before ngOnInit on first load -- and on every subsequent change
- * ngOnInit = called once after the first ngOnChanges -- the right place for initialisation work
- * ngAfterViewInit = called after the component's template and all child components are fully rendered -- for DOM access
- * ngOnDestroy = called just before Angular destroys the component -- the right place for all cleanup
- * Order matters = the hooks fire in a guaranteed order -- knowing that order prevents timing bugs

Now the Technical Terms

- * OnChanges = interface requiring ngOnChanges(changes: SimpleChanges) -- fires when bound input() values change
- * OnInit = interface requiring ngOnInit() -- fires once after first ngOnChanges
- * DoCheck = interface requiring ngDoCheck() -- fires on every change detection cycle -- use sparingly
- * AfterContentInit = fires after ng-content projected content is initialised
- * AfterViewInit = interface requiring ngAfterViewInit() -- fires after the component's view and child views are initialised

- * OnDestroy = interface requiring ngOnDestroy() -- fires just before the component is destroyed
- * SimpleChanges = an object mapping each changed input() property name to a SimpleChange object containing previousValue, currentValue, and firstChange
- * Note on signal inputs = input() signal functions bypass ngOnChanges entirely -- use effect() or computed() to react to signal input changes instead

2. Connection to Prior Concepts

Lifecycle hooks are where every previous Angular topic connects to timing:

- * Topics 20/21 (Subscriptions and takeUntilDestroyed) -- subscriptions are set up in ngOnInit and cleaned up automatically via DestroyRef. takeUntilDestroyed replaces manual ngOnDestroy cleanup
- * Topic 22 (async pipe) -- the async pipe subscribes when the template is first evaluated (after ngOnInit runs) and unsubscribes via its own internal ngOnDestroy
- * Topic 23 (Signals) -- effect() must be called in the constructor because it needs injection context. Signal updates in ngOnInit work fine. ngAfterViewInit is where you access DOM elements. Signal input() functions bypass ngOnChanges -- use effect() to react to signal input changes
- * Topic 24 (OnPush) -- ngDoCheck fires on every change detection cycle -- useful for debugging OnPush. ngOnChanges only fires when the input reference changes -- exactly the same rule as OnPush

Bridging note: "Lifecycle hooks are Angular's event system for a component's own existence. ngOnInit is when you start work, ngOnDestroy is when you stop. Everything between -- subscriptions, HTTP calls, DOM access, Signal effects -- has a correct hook to live in. Using the wrong hook causes timing bugs that are subtle and hard to diagnose."

3. Code Example

Example 1 -- The complete lifecycle -- all hooks in order

```
import {
  Component, OnChanges, OnInit, DoCheck,
  AfterContentInit, AfterContentChecked,
  AfterViewInit, AfterViewChecked, OnDestroy,
  SimpleChanges, inject, input
} from '@angular/core';

@Component({
  selector: 'app-lifecycle-demo',
  standalone: true,
  template: (p){{ title() }}(/p)
})
export class LifecycleDemoComponent implements
  OnChanges, OnInit, DoCheck,
```

```

AfterContentInit, AfterContentChecked,
AfterViewInit, AfterViewChecked, OnDestroy {

// Signal inputs -- NOTE: signal inputs bypass ngOnChanges
// Use @Input() decorator if you need ngOnChanges to fire
title = input('');
userId = input(0);

// 1. CONSTRUCTOR -- runs first -- inject() calls only -- no inputs yet
constructor() {
  console.log('1. constructor -- class created, inject() ready');
  // DO: inject services via inject()
  // DON'T: make HTTP calls, access DOM, read signal inputs
}

// 2. NGONCHANGES -- only fires for @Input() DECORATOR inputs, NOT signal inputs
ngOnChanges(changes: SimpleChanges): void {
  console.log('2. ngOnChanges -- @Input() decorator values changed:', changes);
  if (changes['userId'] and !changes['userId'].firstChange) {
    console.log('userId changed:', changes['userId'].currentValue);
  }
}

// 3. NGONINIT -- runs once -- the right place for real initialisation
ngOnInit(): void {
  console.log('3. ngOnInit -- component ready, inputs available');
  // Signal input values ARE available here
  // DO: HTTP calls, set up subscriptions, signal effects
  // DON'T: access DOM, access viewChild -- not rendered yet
}

// 4. NGDOCHECK -- runs on EVERY change detection cycle -- use sparingly
ngDoCheck(): void {

```

```

// Very rarely needed -- fires very frequently
}

// 5. NGAFTERCONTENTINIT -- runs once after ng-content is projected
ngAfterContentInit(): void {
  console.log('5. ngAfterContentInit -- projected content ready');
}

// 6. NGAFTERCONTENTCHECKED
ngAfterContentChecked(): void { }

// 7. NGAFTERVIEWSINIT -- runs once after template + children rendered
ngAfterViewInit(): void {
  console.log('7. ngAfterViewInit -- DOM and child components ready');
  // viewChild IS available here
  // DO: access DOM elements, initialise third-party libraries
  // DON'T: update template-bound state here without Signal
  // -- causes ExpressionChangedAfterItHasBeenChecked error
}

// 8. NGAFTERVIEWCHECKED
ngAfterViewChecked(): void { }

// 9. NGONDESTROY -- just before component is removed
ngOnDestroy(): void {
  console.log('9. ngOnDestroy -- clean up');
  // With takeUntilDestroyed/DestroyRef -- often empty
}
}

// LIFECYCLE ORDER (first load):
// 1 constructor -> 2 ngOnChanges -> 3 ngOnInit -> 4 ngDoCheck
// -> 5 ngAfterContentInit -> 6 ngAfterContentChecked
// -> 7 ngAfterViewInit -> 8 ngAfterViewChecked

```



```
// ON DESTRUCTION:

// 9 ngOnDestroy
```

Example 2 -- ngOnChanges vs effect() for reacting to input changes

```
// APPROACH A: @Input() decorator + ngOnChanges (still valid)

@Component({
  selector: 'app-user-detail-classic',
  changeDetection: ChangeDetectionStrategy.OnPush,
  standalone: true,
  template: ...
})

export class UserDetailsClassicComponent implements OnChanges {

  @Input({ required: true }) userId!: number;

  user = signal(null);
  isLoading = signal(false);

  private readonly userService = inject(UserService);
  private readonly destroyRef = inject(DestroyRef);

  ngOnChanges(changes: SimpleChanges): void {
    if (changes['userId']) {
      this.loadUser(this.userId);
    }
  }

  private loadUser(id: number): void {
    this.isLoading.set(true);

    this.userService.getUser(id).pipe(
      takeUntilDestroyed(this.destroyRef)
    ).subscribe({
      next: user => { this.user.set(user); this.isLoading.set(false); },
    });
  }
}
```

```

    error: () => { this.isLoading.set(false); }

  });
}
}

// APPROACH B: input() signal + toSignal + toObservable (modern Angular 17+)
@Component({
  selector: 'app-user-detail-modern',
  changeDetection: ChangeDetectionStrategy.OnPush,
  standalone: true,
  template: ...
})
export class UserDetailModernComponent {

  // Signal input -- note: ngOnChanges does NOT fire for signal inputs
  userId = input.required();

  private readonly userService = inject(UserService);

  readonly userData = toSignal(
    toObservable(this.userId).pipe(
      switchMap(id => this.userService.getUser(id).pipe(
        catchError(() => of(null))
      ))
    ),
    { initialValue: null }
  );

  // No ngOnChanges, no ngOnInit, no ngOnDestroy needed
  // toSignal manages subscription via DestroyRef internally
  // switchMap cancels previous HTTP call when userId changes
}

```

Example 3 -- ngAfterViewInit -- DOM and child component access

```
import { Component, AfterViewInit, ViewChild, ElementRef } from '@angular/core';

@Component({
  selector: 'app-chart',
  standalone: true,
  template: `
    (canvas #chartCanvas width="400" height="200")(/canvas)
    (app-chart-legend #legend [data]="chartData" /)
  `
})
export class ChartComponent implements AfterViewInit, OnDestroy {

  private readonly canvasRef = ViewChild.required('chartCanvas');
  private readonly legendRef = ViewChild.required('legend');

  chartData = [];

  private chart: ChartJS | null = null;

  private readonly destroyRef = inject(DestroyRef);

  ngAfterViewInit(): void {
    // Template is fully rendered -- ViewChild available here
    const ctx = this.canvasRef().nativeElement.getContext('2d');

    this.chart = new ChartJS(ctx, { type: 'bar', data: this.chartData });

    this.legendRef().highlight(0);

    this.destroyRef.onDestroy(() => {
      this.chart?.destroy();

      this.chart = null;
    });
  }
}
```

```

ngOnDestroy(): void {

  // destroyRef.onDestroy() above already handles chart cleanup

}

}

```

Angular Example -- Lifecycle hooks in a production component

```

@Component({

  selector: 'app-product-editor',

  changeDetection: ChangeDetectionStrategy.OnPush,

  standalone: true,

  imports: [ReactiveFormsModule],

  template: ...

})

export class ProductEditorComponent

implements OnChanges, OnInit, AfterViewInit, OnDestroy {

  @Input({ required: true }) productId!: number;

  readonly product = signal(null);

  readonly isLoading = signal(false);

  readonly isSaving = signal(false);

  form!: FormGroup;

  private readonly canvasRef = viewChild('previewCanvas');

  private readonly destroyRef = inject(DestroyRef);

  private readonly productService = inject(ProductService);

  private readonly fb = inject(FormBuilder);

  private readonly analytics = inject(AnalyticsService);

  constructor() {

    // effect() needs injection context -- constructor

    effect(() =) {

```

```

const p = this.product();

if (p) { this.analytics.track('product_viewed', { id: p.id }); }

});

}

// ngOnChanges -- fires on first load AND every productId change
ngOnChanges(changes: SimpleChanges): void {

  if (changes['productId']) { this.loadProduct(this.productId); }

}

// ngOnInit -- one-time setup not dependent on @Input values
ngOnInit(): void {

  this.form = this.fb.group({

    name: ['', Validators.required],

    price: [0, [Validators.required, Validators.min(0)]],

    description: ['']

  });

}

// ngAfterViewInit -- DOM ready, viewChild available
ngAfterViewInit(): void {

  const canvas = this.canvasRef();

  if (canvas) { this.initImagePreview(canvas.nativeElement); }

}

ngOnDestroy(): void {

  this.form.reset();

}

private loadProduct(id: number): void {

  this.isLoading.set(true);

  this.productService.getProduct(id).pipe(

    takeUntilDestroyed(this.destroyRef)

```

```

).subscribe({
  next: p => { this.product.set(p); this.isLoading.set(false); },
  error: () => { this.isLoading.set(false); }
});
}
}

```

4. Before and After Refactor

```

// BEFORE -- everything crammed into ngOnInit

@Component({ selector: 'app-user-profile', standalone: true, template: ... })
export class UserProfileComponent implements OnInit, OnDestroy {

  @Input() userId!: number;
  @ViewChild('chart') chartRef!: ElementRef;

  data: Item[] = [];
  private chart: ChartJS | null = null;
  private sub: Subscription;

  private readonly dataService = inject(DataService);

  ngOnInit(): void {

    // WRONG: Accessing @ViewChild -- not rendered yet
    this.chart = new ChartJS(
      this.chartRef.nativeElement, // TypeError: cannot read nativeElement of undefined
      { type: 'bar', data: [] }
    );

    // WRONG: Data load in ngOnInit -- won't re-run if userId input changes
    this.sub = this.dataService.getData(this.userId).subscribe(data => {
      this.data = data;
    });
  }
}

```

```

}

ngOnDestroy(): void {
  this.sub?.unsubscribe(); // partial cleanup -- chart not destroyed
}
}

// AFTER -- right hook for every task

@Component({
  selector: 'app-user-profile',
  changeDetection: ChangeDetectionStrategy.OnPush,
  standalone: true,
  template: ...
})

export class UserProfileComponent
  implements OnChanges, AfterViewInit, OnDestroy {

  @Input({ required: true }) userId!: number;

  private readonly canvasRef = viewChild('chart');

  readonly data = signal([]);
  readonly isLoading = signal(false);

  private chart: ChartJS | null = null;
  private readonly destroyRef = inject(DestroyRef);
  private readonly dataService = inject(DataService);

  // ngOnChanges -- handles BOTH first load AND userId changes
  ngOnChanges(changes: SimpleChanges): void {
    if (changes['userId']) { this.loadData(this.userId); }
  }
}

```

```

// ngAfterViewInit -- DOM is ready, viewChild available

ngAfterViewInit(): void {

  const canvas = this.canvasRef();

  if (canvas) {

    this.chart = new ChartJS(canvas.nativeElement, { type: 'bar', data: { labels: [],
    datasets: [] } });

  }

  this.destroyRef.onDestroy(() => {

    this.chart?.destroy();

    this.chart = null;

  });

}

ngOnDestroy(): void {

  // destroyRef.onDestroy handles chart cleanup

  // takeUntilDestroyed handles subscription cleanup

}

private loadData(userId: number): void {

  this.isLoading.set(true);

  this.dataService.getData(userId).pipe(

    takeUntilDestroyed(this.destroyRef)

  ).subscribe({

    next: items => { this.data.set(items); this.isLoading.set(false); },

    error: () => { this.isLoading.set(false); }

  });

}

}

```

5. Common Mistakes and Misconceptions

1. "Put everything in ngOnInit -- it runs after the component is ready"

ngOnInit runs after the first ngOnChanges -- meaning inputs are available. But the DOM is NOT ready (viewChild is undefined), child components are NOT rendered, and it fires only ONCE -- so it

will not respond to input changes later.

```
// Three common ngOnInit mistakes:

ngOnInit(): void {

  // Mistake 1: DOM access -- not rendered yet
  this.canvasRef().nativeElement.focus(); // undefined

  // Mistake 2: input-driven data that won't reload when input changes
  this.loadData(this.userId); // only runs once

  // Mistake 3: third-party library needing the DOM
  this.map = new GoogleMap(this.mapContainer().nativeElement); // undefined
}

// CORRECT -- each in the right hook:
ngOnChanges(changes: SimpleChanges): void {
  if (changes['userId']) { this.loadData(this.userId); }
}

ngAfterViewInit(): void {
  this.canvasRef().nativeElement.focus();

  this.map = new GoogleMap(this.mapContainer().nativeElement);
}
```

2. "ngOnChanges only fires when the component is created"

ngOnChanges fires on first load AND every time any @Input() decorator value changes.

Important: ngOnChanges only fires for @Input() decorator inputs -- NOT for input() signal functions. For signal inputs, use effect() or toObservable().pipe(switchMap(...)) instead.

3. "Updating state in ngAfterViewInit is fine -- the view is ready"

Angular runs change detection before ngAfterViewInit fires. Updating a template-bound property in ngAfterViewInit creates a mismatch and throws ExpressionChangedAfterItHasBeenCheckedError in dev mode.

```
// Fix 1 -- Promise.resolve() -- defers to next microtask

ngAfterViewInit(): void {

  Promise.resolve().then(() => {
```

```

    this.title.set('Ready');

  });

}

// Fix 2 -- afterNextRender() -- Angular 17+ official solution

constructor() {

  afterNextRender(() => {

    this.title.set('Ready'); // always safe

  });

}

```

6. Do's and Don'ts

Use `ngOnChanges` for data loading driven by `@Input()` decorator inputs

Do NOT use `ngOnInit` alone for input-driven data loading -- won't respond to input changes

Use `ngOnInit` for setup that doesn't depend on input values

Do NOT access `viewChild` in `ngOnInit` -- template not rendered yet

Use `ngAfterViewInit` for DOM access and third-party library initialisation

Do NOT update template-bound state directly in `ngAfterViewInit`

Use `ngOnDestroy` or `destroyRef.onDestroy()` for third-party library cleanup

Do NOT leave `ngOnDestroy` empty for third-party libraries

Implement lifecycle interfaces explicitly -- implements `OnInit`, `OnDestroy`

Do NOT skip implementing interfaces -- a typo in the method name silently never fires

Use `takeUntilDestroyed` / `DestroyRef.onDestroy()`

Do NOT put all cleanup in `ngOnDestroy` -- `DestroyRef` is more composable

7. Debugging Tips

Bug: `viewChild` is undefined in `ngOnInit`

Plain English cause: `ngOnInit` runs before the template is rendered

Fix: Move all `viewChild` access to `ngAfterViewInit`

Bug: Component shows stale data when navigating between items

Plain English cause: Data loading is in `ngOnInit` -- only fires once -- changing the `userId` input does not trigger a reload

Fix: Move data loading to `ngOnChanges` and check `changes['userId']` -- or use `input()` signal with `effect()` or `toObservable().pipe(switchMap(...))`

Bug: `ExpressionChangedAfterItHasBeenCheckedError` in dev mode

Plain English cause: A template-bound property was changed after Angular committed the view -- usually in `ngAfterViewInit`

Fix: Wrap the state update in `Promise.resolve().then(...)` or use `afterNextRender()`

Bug: ngOnChanges not firing on object input mutations

Plain English cause: `ngOnChanges` only fires when the input reference changes -- mutation (same object, new property value) is invisible to it

Fix: Parent must create a new object reference -- `{ ...existingObj, prop: newVal }`

Bug: Memory leak -- third-party library not cleaned up

Plain English cause: Third-party library initialised in `ngAfterViewInit` but `destroy()` never called

Fix: Call the library's cleanup method in `ngOnDestroy` or via `destroyRef.onDestroy()`

8. Real-World Applications

Application 1 -- User Profile with Navigation Between Profiles

```
@Component({
  selector: 'app-user-profile',
  changeDetection: ChangeDetectionStrategy.OnPush,
  standalone: true,
  imports: [DatePipe],
  template: ...
})

export class UserProfileComponent implements OnChanges {

  // Route input -- provided by Angular router (v16+ withComponentInputBinding)
  @Input({ required: true }) userId!: string;

  readonly user = signal(null);
  readonly isLoading = signal(false);

  private readonly destroyRef = inject(DestroyRef);
  private readonly userService = inject(UserService);

  // ngOnChanges -- fires on FIRST LOAD and on EVERY userId change
  ngOnChanges(changes: SimpleChanges): void {
    if (changes['userId']) {
      this.user.set(null);
      this.isLoading.set(true);

      this.userService.getUser(Number(this.userId)).pipe(
```

```

takeUntilDestroyed(this.destroyRef)

).subscribe({

next: user => { this.user.set(user); this.isLoading.set(false); },

error: () => { this.isLoading.set(false); }

});

}

}

}

```

Application 2 -- Interactive Map Component

```

@Component({

  selector: 'app-location-map',

  changeDetection: ChangeDetectionStrategy.OnPush,

  standalone: true,

  template: (div #mapContainer class="map-container" style="height: 400px")(/div)

})

export class LocationMapComponent implements OnChanges, AfterViewInit {

  @Input() latitude: number = 51.5074;

  @Input() longitude: number = -0.1278;

  @Input() zoom: number = 13;

  private readonly mapContainer = viewChild.required('mapContainer');

  private readonly destroyRef = inject(DestroyRef);

  private map: LeafletMap | null = null;

  private marker: LeafletMarker | null = null;

  ngAfterViewInit(): void {

    this.map = L.map(this.mapContainer().nativeElement).setView(

      [this.latitude, this.longitude], this.zoom

    );

    L.tileLayer('https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png').addTo(this.map);

```

```

    this.marker = L.marker([this.latitude, this.longitude]).addTo(this.map);

    this.destroyRef.onDestroy(() => {
      this.map?.remove();
      this.map = null;
      this.marker = null;
    });
  }

  ngOnChanges(changes: SimpleChanges): void {
    if (!this.map) return; // map not initialised yet

    if (changes['latitude'] || changes['longitude']) {
      const latLng = [this.latitude, this.longitude];
      this.map.setView(latLng, this.zoom);
      this.marker?.setLatLng(latLng);
    }
  }
}

```

9. Practice Exercises

Exercise 1 -- Beginner

Build a `CountdownTimerComponent` that takes seconds via `@Input()`. It should start counting down when initialised, update every second using `setInterval`, show the remaining time, show "Time's up!" at zero, and restart correctly when the seconds input changes. Use the correct lifecycle hook for starting the interval, correctly restart when input changes, and clean up the interval via `destroyRef.onDestroy()`.

Exercise 2 -- Intermediate

Build a `DataTableComponent` with two `@Input()` values -- `endpoint: string` and `pageSize: number`. Load data in the correct lifecycle hook, reload when endpoint changes, use `ViewChild()` to access a table element and measure its width in `ngAfterViewInit`, store the width in a `Signal`, and apply conditional styling when the table is too narrow.

Exercise 3 -- Advanced

Build a `VideoPlayerComponent` wrapping a third-party video player library. The component receives `videoUrl` and `autoPlay` via `input()` signal functions. Use `effect()` and `toObservable().pipe(switchMap(...))` instead of `ngOnChanges`. Handle the case where the signal fires before `ngAfterViewInit`. Clean up the player via `destroyRef.onDestroy()`. Document exactly why each lifecycle hook and signal pattern was chosen.

10. Key Takeaways

The Lifecycle Order:

FIRST LOAD: constructor -> ngOnChanges -> ngOnInit -> ngDoCheck -> ngAfterContentInit -> ngAfterContentChecked -> ngAfterViewInit -> ngAfterViewChecked

EVERY CHANGE DETECTION CYCLE (after first): ngOnChanges (if @Input() changed) -> ngDoCheck -> ngAfterContentChecked -> ngAfterViewChecked

DESTRUCTION: ngOnDestroy

The 80/20 Hooks:

- * constructor: inject() only -- no inputs, no DOM
- * ngOnChanges: @Input()-driven data loading -- first load + every change
- * ngOnInit: one-time setup -- form creation, non-input subscriptions
- * ngAfterViewInit: DOM + viewChild access -- third-party library init
- * ngOnDestroy: explicit cleanup -- third-party libs, anything DestroyRef missed

Quick Reference -- Which Hook for What:

- * inject() services: constructor (not anywhere else)
- * Load data based on @Input(): ngOnChanges (not ngOnInit -- won't re-run)
- * One-time form/subscription setup: ngOnInit (not constructor -- inputs not ready)
- * Access viewChild / DOM element: ngAfterViewInit (not ngOnInit -- not rendered)
- * Third-party library init: ngAfterViewInit (not ngOnInit -- no DOM yet)
- * Subscription cleanup: takeUntilDestroyed (not manual ngOnDestroy unsubscribe)
- * Third-party library cleanup: ngOnDestroy / destroyRef.onDestroy()

One-Line Memory Aid: constructor = hired (inject only). ngOnChanges = briefed (inputs ready, fires every time). ngOnInit = first day (setup, once). ngAfterViewInit = office access (DOM ready). ngOnDestroy = leaving (clean up). Wrong hook = subtle timing bug.

11. Thought-Provoking Question and Answer

Angular 17+ introduced input() signal functions as a replacement for @Input() decorators. If inputs are now Signals, ngOnChanges becomes redundant -- you can use computed() or effect() to react to input changes instead. Does this mean ngOnChanges is being deprecated, and what is the cleanest way to react to input changes in modern Angular?

ngOnChanges is NOT deprecated -- Angular team confirmed it remains supported.

BUT with signal inputs (input()):

- * ngOnChanges does NOT fire for signal inputs
- * Signal inputs bypass the ngOnChanges system entirely

The migration path:

- * @Input() userId: number -> userId = input.required(number)
- * ngOnChanges + if(changes.userId) + subscribe -> toSignal(toObservable(this.userId).pipe(switchMap(id => getUser(id))))

```
// MODERN WAY -- input() Signal + toObservable + switchMap

@Component({ standalone: true, template: '...' })
```

```

export class NewComponent {

  readonly userId = input.required();

  private readonly userService = inject(UserService);

  // Handles cancellation, subscription management, OnPush, null state
  readonly userData = toSignal(
    toObservable(this.userId).pipe(
      switchMap(id => this.userService.getUser(id).pipe(
        catchError(() => of(null))
      ))
    ),
    { initialValue: null }
  );

  // No ngOnChanges, no ngOnInit, no ngOnDestroy needed
}

```

Practical rule: In modern Angular 17+, prefer `input()` signal functions for new components. `toSignal(toObservable(this.inputSignal).pipe(switchMap(...)))` is the most complete pattern -- it handles cancellation, subscription management, and `OnPush` automatically.

12. Related Topics to Learn Next

- * `input()` and `output()` signal functions (Topic 26) -- signal inputs `input()`, required inputs, transforms, aliases
- * Services and Dependency Injection (Topic 27) -- services injected via `inject()` in constructor, used in lifecycle hooks
- * Angular Template Syntax (Topic 28) -- `@if`, `@for`, `@switch` -- what gets rendered during the `AfterViewInit` phase
- * Angular Reactive Forms (Topic 33) -- forms initialised in `ngOnInit`, form values in `ngOnChanges`

13. Updated Learning Tracker

Topics 1-25: All through Component Lifecycle Hooks -- COMPLETE

Next: Topic 26 -- `input()` and `output()` in Depth

Topic 26 -- input() and output() -- Component Communication in Depth

0. Difficulty and Priority Rating

Difficulty: Beginner-Intermediate -- the basic syntax is simple; the depth comes from understanding required inputs, transforms, aliases, the difference between decorator and signal approaches, and the correct immutability patterns for passing data between components

Angular Relevance: Critical -- component communication is the primary mechanism for building Angular apps; every parent-child relationship uses inputs and outputs; getting them wrong causes silent bugs, stale data, and broken templates

1. Explanation

The Manager and Employee Analogy

Imagine a company with a manager (parent component) and an employee (child component):

`input()` = the manager giving the employee their work brief. The manager decides what information the employee receives -- the employee cannot ask for it themselves. The brief arrives before the employee starts work. If the brief changes, the employee's display board updates automatically.

`output()` = the employee's reporting system. When they finish a task, complete a form, or discover something important, they send a report UP to the manager. The manager decides what to do with the report.

Manager (Parent) gives brief (input): `[user]="selectedUser()"`

Employee receives it: `readonly user = input.required(User)`

Employee reports back: `readonly userSaved = output(User)`

Manager handles report: `(userSaved)="onUserSaved($event)"`

The USB Port Analogy

Think of `input()` as a USB port on a device. The device does not reach out to get power -- power is pushed INTO it. `output()` is like a speaker jack -- sound comes OUT of the device when something internal triggers it.

In Plain English

- * `input()` = a Signal property that receives data pushed from a parent -- the parent controls what goes in -- read with `()`
- * `output()` = a lightweight emitter that sends data up to the parent -- the child controls when it emits -- no `EventEmitter` needed
- * Unidirectional data flow = data flows DOWN via inputs, events flow UP via outputs -- Angular's fundamental design principle
- * `input.required(T)()` = the parent MUST provide this value -- Angular enforces it at build time
- * `computed()` from inputs = derive values from `input()` signals automatically -- replaces `ngOnChanges` for derived state
- * `@Input()/@Output()` decorators = the older decorator-based approach -- still valid, still supported, common in existing codebases

Now the Technical Terms

- * `input(T)(defaultValue?)` = creates a read-only `InputSignal(T)` -- parent binds with `[property]="value"` -- read with `this.property()`
- * `input.required(T)()` = Angular enforces the parent must provide this -- compile error if missing
- * `input(value, { transform })` = automatically transforms the incoming value before it reaches the signal
- * `input(value, { alias: 'publicName' })` = parent uses a different name than the internal property name
- * `output(T)()` = creates an `OutputEmitterRef(T)` -- parent listens with `(eventName)="handler($event)"` -- emit with `.emit(value)`
- * `$event` = the Angular template variable that receives whatever was emitted
- * `@Input()` decorator = older approach -- marks a class property as an input binding -- `ngOnChanges` fires when it changes
- * `@Output() + EventEmitter(T)` = older approach -- `.emit(value)` triggers all parent listeners

2. Connection to Prior Concepts

- * Topic 7 (Pass by Reference) -- when a parent passes an object via `input()`, the child receives a reference. If the child's logic mutates that object, the parent sees the mutation. Always treat input values as read-only
- * Topic 24 (OnPush Change Detection) -- `OnPush` checks if `input()` values changed by reference. Same reference means no check. This is why immutable updates are mandatory in `OnPush` component trees
- * Topic 25 (Lifecycle Hooks) -- `input()` signal values are available from construction. `computed()` and `effect()` replace `ngOnChanges` for reacting to signal input changes
- * Topic 23 (Signals) -- `input()` returns a `Signal` -- use `computed()` to derive values and `effect()` to react to changes
- * Topic 13 (TypeScript Interfaces) -- every `input()` should have a TypeScript type -- `input.required(User)()` not `input.required(any)()`

Bridging note: "input() is the USB port -- data pushed in from outside. output() is the speaker -- signals sent out when something happens inside. The parent controls what goes in and decides what to do with what comes out. The child defines the shape of the port and what it reports."

3. Code Example

Example 1 -- Modern input() and output() -- Angular 17+

```
import { input, output, computed, effect } from '@angular/core';

// CHILD COMPONENT -- modern signal-based

@Component({
  selector: 'app-user-card',
  standalone: true,
  template: `
    (div [class.selected]="isSelected()")
    (h3){ { user().name } }(/h3)
  `
})
```

```

    (p){ { user().email } }(/p)

    (span){ { user().role } }(/span)

    (p){ { displayName() } }(/p)

    (button (click)="onSelect()")Select(/button)

    (button (click)="onDelete()")Delete(/button)

  (/div)
  `
})

export class UserCardComponent {

  // input() -- receives data FROM parent -- returns a Signal
  readonly user = input.required(); // required -- parent must provide
  readonly isSelected = input(false); // optional with default

  // computed() -- derives from signal inputs -- updates automatically
  readonly displayName = computed(() =>
    `${this.user().name} (${this.user().role})`
  );

  // output() -- sends events TO parent -- no EventEmitter needed
  readonly userSelected = output();
  readonly userDeleted = output();
  readonly dismissed = output();

  onSelect(): void { this.userSelected.emit(this.user()); }
  onDelete(): void { this.userDeleted.emit(this.user().id); }
}

// PARENT COMPONENT

@Component({
  selector: 'app-user-list',
  standalone: true,
  imports: [UserCardComponent],

```

```

template: `
@for (user of users(); track user.id) {

  (app-user-card

  [user]="user"

  [isSelected]="selectedId() === user.id"

  (userSelected)="onUserSelected($event)"

  (userDeleted)="onUserDeleted($event)" /)

}
`

})

export class UserListComponent {

  private readonly userService = inject(UserService);

  readonly users = toSignal(this.userService.getUsers(), { initialValue: [] });

  readonly selectedId = signal(null);

  onUserSelected(user: User): void { this.selectedId.set(user.id); }

  onUserDeleted(id: number): void { this.userService.deleteUser(id); }

}

```

Example 2 -- input() options -- required, transform, alias

```

import { input, output, numberAttribute, booleanAttribute } from '@angular/core';

@Component({ selector: 'app-product-card', standalone: true, template: '...' })

export class ProductCardComponent {

  // required -- parent MUST provide this

  readonly product = input.required();

  // transform -- automatically converts incoming value

  // Parent passes string: [price]="29.99" -- Component receives number: 29.99

  readonly price = input(0, { transform: (v: string) => parseFloat(v) });

  // Angular built-in transforms

```

```

readonly quantity = input(0, { transform: numberAttribute }); // string -> number
readonly isActive = input(false, { transform: booleanAttribute }); // string -> boolean
// Essential for HTML attribute bindings -- HTML attributes are always strings

// alias -- parent uses different name than internal property
readonly productData = input.required({ alias: 'item' });
// Parent template: [item]="selectedProduct"
// Internal code: this.productData().name

// computed from signal inputs -- automatic
readonly displayPrice = computed(() => ${this.price().toFixed(2)});
readonly isAvailable = computed(() => this.isActive() and this.product().stock > 0);
}

```

Example 3 -- @Input() / @Output() decorator approach -- still valid for existing code

```

@Component({
  selector: 'app-settings-panel',
  standalone: true,
  imports: [ReactiveFormsModule],
  template: ...
})
export class SettingsPanelComponent implements OnChanges, OnInit {

  @Input({ required: true }) settings!: UserSettings;
  @Input() userId!: number;

  @Output() settingsSaved = new EventEmitter();
  @Output() settingsChanged = new EventEmitter();

  form!: FormGroup;

  localSettings: UserSettings | null = null;

  private readonly fb = inject(FormBuilder);

  ngOnInit(): void {

```

```

this.form = this.fb.group({
  theme: ['light'],
  fontSize: [14],
  notifications: [true]
});
}

// ngOnChanges -- fires for @Input() decorator inputs
// (does NOT fire for input() signal functions)
ngOnChanges(changes: SimpleChanges): void {
  if (changes['settings'] and this.settings) {
    // Always spread @Input() before working with it -- never mutate directly
    this.localSettings = { ...this.settings };
    this.form?.patchValue(this.localSettings);
  }
}

onSave(): void {
  if (this.form.invalid || !this.localSettings) return;
  this.settingsSaved.emit({ ...this.localSettings, ...this.form.value }); // immutable
}
}

```

Angular Example -- Modern signal-based component with all patterns

```

@Component({
  selector: 'app-user-editor',
  changeDetection: ChangeDetectionStrategy.OnPush,
  standalone: true,
  imports: [ReactiveFormsModule],
  template: `
    @if (isLoading()) { (div>Loading...{/div) }
    @else {

```

```

(h2)Editing: {{ user().name }}(/h2)

(p){{ displayName() }}(/p)

(form [formGroup]="form" (ngSubmit)="onSave()")

  (input formControlName="name")

  (input formControlName="email")

  (button type="submit" [disabled]="form.invalid || isSaving()")

  {{ isSaving() ? 'Saving...' : 'Save' }}

(/button)

(button type="button" (click)="onCancel()")Cancel(/button)

(/form)

}
`

})

export class UserEditorComponent {

  readonly user = input.required();

  readonly mode = input('edit');

  readonly displayName = computed(() => `${this.user().name} -- ${this.user().role}`);

  readonly isEditing = computed(() => this.mode() === 'edit');

  readonly saved = output();

  readonly cancelled = output();

  readonly isSaving = signal(false);

  readonly isLoading = signal(false);

  readonly form = new FormGroup({

    name: new FormControl('', Validators.required),

    email: new FormControl('', [Validators.required, Validators.email]),

  });

  private readonly analytics = inject(AnalyticsService);

```

```

constructor() {

effect(() => {

const u = this.user();

this.form.patchValue({ name: u.name, email: u.email });

this.analytics.track('user_editor_opened', { userId: u.id });

});

}

onSave(): void {

if (this.form.invalid) return;

this.isSaving.set(true);

this.saved.emit({ ...this.user(), ...this.form.value });

this.isSaving.set(false);

}

onCancel(): void { this.cancelled.emit(); }

}

```

4. Before and After Refactor

```

// BEFORE -- two common mistakes in one component

@Component({

selector: 'app-user-editor',

changeDetection: ChangeDetectionStrategy.OnPush,

standalone: true,

template: (p){ { user?.name } }(/p)

})

export class UserEditorComponent implements OnInit {

@Input() user!: User; // no required, no type safety

@Output() saved = new EventEmitter(); // untyped

ngOnInit(): void {

```

```

// Mistake 1: only fires once -- won't reload if user input changes

this.loadAdditionalData(this.user.id);

}

saveChanges(newName: string): void {

// Mistake 2: mutating the @Input() directly

this.user.name = newName; // mutates parent's object

this.saved.emit(this.user);

}

}

// AFTER -- modern signal-based, immutable, reactive

@Component({

selector: 'app-user-editor',

changeDetection: ChangeDetectionStrategy.OnPush,

standalone: true,

template: `

@if (user()) {

(p){{ user().name }}(/p)

(button (click)="saveChanges('Bob')")Save(/button)

}

`

})

export class UserEditorComponent {

readonly user = input.required();

readonly saved = output();

private readonly userService = inject(UserService);

constructor() {

// effect() reacts to EVERY user() signal change -- not just first load

```



```

effect(() => {
  const u = this.user();

  if (u) { this.userService.loadAdditionalData(u.id); }

});

}

saveChanges(newName: string): void {
  // Immutable update -- never mutate the input signal value

  const updatedUser = { ...this.user(), name: newName }; // new object

  this.saved.emit(updatedUser); // emit to parent -- parent decides
}

}

```

5. Common Mistakes and Misconceptions

1. "I can mutate my input value -- I am just updating the same data"

Mutating an input value is like the employee rewriting the manager's original brief. Always treat input values as read-only. Copy with spread before modifying, and emit changes back up via output().

```

// WRONG: Mutating input value

readonly config = input.required();

applyTheme(theme: string): void {

  this.config().theme = theme; // mutates the object inside the signal

}

// CORRECT: Immutable: copy first, emit back

readonly config = input.required();

readonly configChanged = output();

applyTheme(theme: string): void {

  const updated = { ...this.config(), theme };

  this.configChanged.emit(updated); // parent decides

}

```

2. "If input data comes from an HTTP call, I can just use ngOnInit"

ngOnInit fires exactly once. If the component is reused by Angular's router with a different input, ngOnInit will not fire again. With input() signals and effect(), the reaction happens automatically on every input change.

```

// WRONG: ngOnInit only -- won't reload when userId signal changes

ngOnInit(): void {

  this.loadUser(this.userId()); // only runs once

}

// CORRECT: effect() -- re-runs whenever userId signal changes

constructor() {

  effect(() => {

    this.loadUser(this.userId()); // runs on first value AND every change

  });

}

// EVEN BETTER: toSignal + toObservable + switchMap

readonly userData = toSignal(

  toObservable(this.userId).pipe(

    switchMap(id => this.userService.getUser(id).pipe(catchError(() => of(null))))

  ),

  { initialValue: null }

);

```

3. "Output events automatically update the parent's state"

output() just emits a value -- it does nothing else. The parent decides what to do with it. If the parent has no handler, the emitted value disappears silently.

```

// CORRECT: Parent handler explicitly updates state

onUserSaved(updatedUser: User): void {

  this.users.update(users =>

    users.map(u => u.id === updatedUser.id ? updatedUser : u)

  );

}

```

6. Do's and Don'ts

Use input.required(T()) for inputs that must always be provided -- compile error if missing

Do NOT leave required inputs without enforcement

Use effect() to react to input() signal changes -- fires on every change automatically

Do NOT use `ngOnInit` alone for input-driven data loading

Treat input values as read-only -- copy with `spread` before modifying

Do NOT mutate input signal values directly in the child

Type every `output()` -- `output(User)()` not `output()`

Do NOT use untyped outputs

Use `input()` and `output()` functions for new Angular 17+ components

Do NOT mix `@Input()` decorator and `input()` function on the same property

Use `input(0, { transform: numberAttribute })` for numeric inputs from HTML attributes

Do NOT manually parse string inputs

7. Debugging Tips

Bug: `input()` value is undefined initially

Plain English cause: `input()` without a default value and `input.required()` without a parent binding

Fix: Use `input.required(T)()` -- Angular will error at build time if parent forgets

Bug: Component shows stale data when navigating between items

Plain English cause: Data loading is in `ngOnInit` -- fires only once

Fix: Move data loading to `effect(() => { this.loadUser(this.userId()); })` in the constructor, or use `toSignal(toObservable(this.userId).pipe(switchMap(...)))`

Bug: Parent state unexpectedly changed -- nobody called the update method

Plain English cause: Child mutated the input value object directly -- since objects are passed by reference, the parent sees the mutation

Fix: In the child, always `const local = { ...this.input() }` before modifying

Bug: `(outputName)` handler not firing in parent

Plain English cause: Either `output()` property is missing, event name doesn't match, or `.emit()` is never called

Fix: Verify `output()` is declared. Check template event name matches property name. Verify `.emit()` is called.

8. Real-World Applications

Application 1 -- Data Table with Inline Editing

```
@Component({
  selector: 'app-user-row',
  changeDetection: ChangeDetectionStrategy.OnPush,
  standalone: true,
  imports: [ReactiveFormsModule],
  template: `
    @if (!isEditing()) {
      (td){ { user().name } }(/td)
    }
```

```

(td)

(button (click)="startEdit()")Edit(/button)

(button (click)="onDelete()")Delete(/button)

(/td)

} @else {

(td)(input [formControl]="nameControl")(/td)

(td)

(button (click)="onSave()" [disabled]="isSaving()")

{{ isSaving() ? 'Saving...' : 'Save' }}

(/button)

(button (click)="cancelEdit()")Cancel(/button)

(/td)

}

`

}))

export class UserRowComponent {

  readonly user = input.required();

  readonly isSaving = input(false);

  readonly userUpdated = output();

  readonly userDeleted = output();

  readonly isEditing = signal(false);

  readonly nameControl = new FormControl('');

  readonly emailControl = new FormControl('');

  constructor() {

    effect(() => {

      if (!this.isSaving()) { this.isEditing.set(false); }

    });

  }

```

```

startEdit(): void {
  this.nameControl.setValue(this.user().name);
  this.emailControl.setValue(this.user().email);
  this.isEditing.set(true);
}

onSave(): void {
  this.userUpdated.emit({
    ...this.user(),
    name: this.nameControl.value ?? this.user().name,
    email: this.emailControl.value ?? this.user().email
  });
}

cancelEdit(): void { this.isEditing.set(false); }

onDelete(): void { this.userDeleted.emit(this.user().id); }
}

```

Application 2 -- Multi-Step Form with Signal Inputs

```

@Component({
  selector: 'app-address-step',
  standalone: true,
  imports: [ReactiveFormsModule],
  template: `
    (h2){ { stepTitle() } }(/h2)
    (form [formGroup]="form" (ngSubmit)="onNext()")
    (input formControlName="line1" placeholder="Address line 1")
    (input formControlName="city" placeholder="City")
    (input formControlName="postcode" placeholder="Postcode")
    (button type="submit" [disabled]="form.invalid")Next(/button)
    (button type="button" (click)="onBack()")Back(/button)
  (/form)

```

```

    })

    export class AddressStepComponent {

        readonly stepNumber = input.required();

        readonly initialData = input({});

        readonly stepTitle = computed(() =>
            Delivery Address (Step ${this.stepNumber()} of 3)
        );

        readonly completed = output();

        readonly back = output();

        readonly form = new FormGroup({
            line1: new FormControl('', Validators.required),
            city: new FormControl('', Validators.required),
            postcode: new FormControl('', Validators.required)
        });

        constructor() {
            effect(() => {
                const data = this.initialData();

                if (data and Object.keys(data).length > 0) {
                    this.form.patchValue(data);
                }
            });
        }

        onNext(): void {
            if (this.form.invalid) return;

            this.completed.emit(this.form.value);
        }
    }

```

```
onBack(): void { this.back.emit(); }  
  
}
```

9. Practice Exercises

Exercise 1 -- Beginner

Build a `RatingComponent` that displays 5 stars. It receives `rating = input.required(number)` (1-5) and emits `ratingChanged = output(number)` when a star is clicked. The component should highlight stars up to the current rating using `computed()`. When a star is clicked, emit the new rating to the parent. Verify that clicking a star updates the parent's display.

Exercise 2 -- Intermediate

Build a `FilterPanelComponent` that receives `products = input.required(Product[])` and `categories = input.required(string[])`. It should have local filter state (selected category, price range, in-stock only) as signals. Use `computed()` to derive the filtered product list automatically. Emit `filtered = output(Product[])` whenever the filtered list changes. Use `OnPush` change detection.

Exercise 3 -- Advanced

Build a generic `DataTableComponent` using `input()` and `output()` signal functions. Accept: `data = input.required(T[])`, `columns = input.required(TableColumn(T)[])`, `pageSize = input(10)`, `sortable = input(true)`. Emit: `rowClick = output(T)`, `sortChanged = output(SortEvent(T))`, `pageChanged = output(number)`. Use `computed()` for sorted and paginated data. Use `effect()` to reset to page 1 when data input changes.

10. Key Takeaways

Core Concepts in Plain English:

- * `input()` = USB port -- data pushed IN from parent -- read with `()` -- treat as read-only
- * `output()` = speaker jack -- signals sent OUT to parent -- no `EventEmitter` needed
- * Unidirectional flow = data down via inputs, events up via outputs
- * Never mutate input values = spread first, work on local copy, emit changes back
- * `effect()` not `ngOnInit` = for input-driven data loading -- fires on every signal change
- * `computed()` from inputs = automatic derived state -- replaces `ngOnChanges` pattern
- * `@Input()` + `ngOnChanges` = still valid -- use in existing codebases and when `SimpleChanges` is needed

`input()` Options -- Quick Reference:

```
// Basic signal input  
  
readonly value = input();  
  
// Required -- compile error if parent does not provide  
  
readonly value = input.required();  
  
// With default  
  
readonly value = input('default value');
```

```
// With transform

readonly count = input(0, { transform: numberAttribute });

readonly flag = input(false, { transform: booleanAttribute });


// With alias -- parent binds [publicName], code reads this.internalName()

readonly internalName = input.required({ alias: 'publicName' });


// output

readonly myEvent = output(); // typed

readonly dismissed = output(); // no data
```

Decorator vs Signal approach:

- * `@Input()` decorator / `input()` signal: Style Decorator/Function, Available Angular 2+ / Angular 17+, React to changes `ngOnChanges` / `computed()` or `effect()`, Recommended for Existing codebases / New projects

- * `@Output()` + `EventEmitter` / `output()`: Style Decorator/Function, Available Angular 2+ / Angular 17+, Recommended for Existing codebases / New projects

One-Line Memory Aid: `input()` = USB port -- data pushed in from parent, read with `()`, treat as read-only. `output()` = speaker -- emit when something happens. Copy before modifying. Use `effect()` not `ngOnInit`. Parent controls data; child controls events.

11. Thought-Provoking Question and Answer

Angular 17+ introduced `input()` signal functions as alternatives to `@Input()` decorators. Does this mean `ngOnChanges` is being deprecated, and what is the architectural difference between the two approaches?

`ngOnChanges` approach -- notification letter system: You open the letter, read which inputs changed, and decide what to do. Pull-based.

Signal input approach -- live display board: You describe a formula that reads from the board. When the board updates, the formula automatically recalculates. Declarative.

```
// MODERN: input() Signal + toObservable + switchMap

@Component({ standalone: true, template: '...' })

export class NewComponent {

  readonly userId = input.required();

  readonly filter = input('all');


  private readonly userService = inject(UserService);


  readonly data = toSignal(
```



```

combineLatest([
  toObservable(this.userId),
  toObservable(this.filter)
]).pipe(
  switchMap(([id, filter]) =>
    this.userService.loadData(id, filter).pipe(catchError(() => of([])))
  )
),
{ initialValue: [] }
);

readonly displayFilter = computed(() =>
  this.filter() === 'all' ? 'Showing all' : Filtered by: ${this.filter()}
);
}

```

ngOnChanges is NOT deprecated -- Angular team confirmed ongoing support.

When @Input() + ngOnChanges is still right:

- * Maintaining Angular 14/15/16 codebases
- * You need previousValue for comparison
- * Working in a codebase using decorator-based inputs consistently

When input() + effect/computed is better:

- * New Angular 17+ projects
- * Multiple derived values
- * Full signal-based architecture
- * Want cancellable async reactions via toObservable + switchMap

Practical rule: For new Angular 17+ projects, prefer input() functions. The key architectural difference: ngOnChanges is you checking a notification letter; input() signals are a live board that formulas watch automatically.

12. Related Topics to Learn Next

- * Services and Dependency Injection (Topic 27) -- services are the alternative to deeply nested input passing
- * Angular Template Syntax (Topic 28) -- property binding [prop], event binding (event)
- * Angular Reactive Forms (Topic 33) -- forms receive data via inputs and emit via outputs
- * viewChild() and contentChild() (Topic 32) -- accessing child component instances directly
- * Content Projection ng-content (Topic 31) -- a different kind of input

13. Updated Learning Tracker

Topics 1-26: All through input() and output() -- COMPLETE

Next: Topic 27 -- Services and Dependency Injection